

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT & CÔNG NGHỆ



ISO 9001:2015

**XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ GIẢNG
VIÊN THEO DÕI TIẾN ĐỘ HỌC TẬP VÀ CẢNH BÁO
SINH VIÊN CÓ NGUY CƠ THÔI HỌC**

KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

VĨNH LONG, NĂM 2026

TRƯỜNG ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT & CÔNG NGHỆ

**XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ GIẢNG
VIÊN THEO DÕI TIẾN ĐỘ HỌC TẬP VÀ CẢNH BÁO
SINH VIÊN CÓ NGUY CƠ THÔI HỌC**

**KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN**

Sinh viên: **Nguyễn Đình Nhật Huy**

Lớp: **DA22TTB**

MSSV: **110122223**

GVHD: **ThS. Trịnh Quốc Việt**

VĨNH LONG, NĂM 2026

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn chân thành và sâu sắc nhất đến Thầy – Thạc sĩ Trịnh Quốc Việt, Giảng viên Khoa Kỹ thuật & Công nghệ, Đại học Trà Vinh , người đã trực tiếp hướng dẫn em trong suốt quá trình thực hiện đồ án tốt nghiệp.

Trong suốt thời gian thực hiện đề tài, Thầy không chỉ là người định hướng về mặt chuyên môn mà còn là người luôn đồng hành, động viên và hỗ trợ em vượt qua những khó khăn, vướng mắc cả trong học thuật lẫn trong quá trình triển khai thực tế. Thầy đã tận tình góp ý, đưa ra những lời khuyên xác đáng, đồng thời luôn tạo điều kiện thuận lợi để em có thể hoàn thiện đề tài một cách tốt nhất.

Bằng kiến thức sâu rộng, tinh thần trách nhiệm cao và sự tận tụy với sinh viên, Thầy đã giúp em không chỉ hiểu rõ hơn về chuyên môn mà còn học hỏi thêm nhiều kỹ năng thực tiễn và cách tư duy logic trong giải quyết vấn đề. Những buổi trao đổi, phản biện và định hướng từ Thầy là những trải nghiệm quý báu, giúp em hoàn thiện hơn từng bước trong quá trình làm đồ án.

Em cảm thấy vô cùng may mắn và trân trọng khi được Thầy hướng dẫn. Những kiến thức và kinh nghiệm học được từ Thầy chắc chắn sẽ là hành trang quý giá để em áp dụng và phát triển trong chặng đường sắp tới.

Một lần nữa, em xin gửi đến Thầy lời cảm ơn chân thành, kính chúc Thầy luôn mạnh khỏe, công tác tốt và tiếp tục truyền cảm hứng, kiến thức cho nhiều thế hệ sinh viên tiếp theo.

Sinh viên thực hiện

Nguyễn Đình Nhật Huy

NHẬN XÉT

(Của giảng viên hướng dẫn trong đồ án, khóa luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP

(Của giảng viên hướng dẫn)

Họ và tên sinh viên: Nguyễn Đình Nhật Huy MSSV: 110122223

Ngành: Công nghệ thông tin Khóa: 2022

Tên đề tài: Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học.

Họ và tên Giáo viên hướng dẫn: Trịnh Quốc Việt

Chức danh: Giảng viên Học vị: Thạc sĩ

NHẬN XÉT

1. Nội dung đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. Ưu điểm:

.....

.....

.....

.....

3. Khuyết điểm:

.....

.....

.....
.....
4. Điểm mới đề tài:

.....
.....
.....
.....
5. Giá trị thực trên đề tài:

.....
.....
.....
.....
6. Đề nghị sửa chữa bổ sung:

.....
.....
.....
.....
7. Đánh giá:

.....
.....
.....
.....
Vĩnh Long, ngày..... tháng.....năm 20...
Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)

NHẬN XÉT

(Của giảng viên chấm trong đồ án, khóa luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP

(Của cán bộ chấm đồ án, khóa luận)

Họ và tên người nhận xét:

Chức danh:..... Học vị:.....

Chuyên ngành:.....

Cơ quan công tác:.....

Họ và tên sinh viên: Nguyễn Đình Nhật Huy

Tên đề tài đồ án, khóa luận tốt nghiệp: Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học.

I. Ý KIẾN NHẬN XÉT

1. Nội dung:

.....
.....
.....
.....
.....
.....
.....
.....

2. Điểm mới các kết quả của đồ án, khóa luận:

.....
.....
.....

3. Ứng dụng thực tế:

.....
.....
.....
.....
.....
.....

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

(Các câu hỏi của giáo viên phản biện)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đồ án khóa luận tốt nghiệp)

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngàytháng..... năm 20...

Người nhận xét

(Ký & ghi rõ họ tên)

MỤC LỤC

LỜI CẢM ƠN	4
NHẬN XÉT	5
NHẬN XÉT	8
MỤC LỤC	11
DANH MỤC HÌNH.....	14
DANH MỤC BẢNG.....	15
CHƯƠNG 1. MỞ ĐẦU.....	18
1.1. Lý do chọn đề tài	18
1.2. Mục tiêu.....	2
1.3. Nội dung nghiên cứu	3
1.4. Đối tượng và phạm vi nghiên	3
1.5. Phương pháp nghiên cứu.....	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT.....	6
2.1. Tổng quan về hệ thống quản lý học tập Moodle	6
2.1.1 Kiến trúc Moodle	6
2.1.2 Moodle Web Services.....	6
2.2. Khái niệm chatbot và ứng dụng trong giáo dục	7
2.2.1 Phân loại chatbot	7
2.2.2 Vai trò của chatbot trong cảnh báo sớm	8
2.3. Xử lý ngôn ngữ tự nhiên trong chatbot.....	8
2.3.1 Nhận diện ý định	8
2.3.2 Trích xuất thực tế	9
2.4. Trí tuệ nhân tạo và mô hình ngôn ngữ lớn trong chatbot	9
2.4.1 Ứng dụng AI trong phân tích học tập.....	9
2.4.2 Kết hợp luật và AI	10
2.5. Cơ sở lý thuyết về phát hiện sinh viên có nguy cơ thôi học.....	10
2.5.1 Các chỉ số đánh giá nguy cơ.....	10

2.5.2 Phân loại công cụ đánh giá mã nguồn.....	11
2.5.3 Quy trình đánh giá mã nguồn.....	11
2.5.4 Kết hợp công cụ đánh giá mã nguồn với GitHub Actions	13
2.6. Các công nghệ sử dụng trong hệ thống	15
2.6.1 Ngôn ngữ lập trình Java	15
2.6.2 Spring Boot Framework.....	15
2.6.3 MySQL.....	16
2.6.4 Quy trình hiển thị thống kê đóng góp	17
CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU	18
3.1. Mô tả bài toán thực tế.....	18
3.2. Phân tích yêu cầu người dùng.....	19
3.2.1 Xác định tác nhân (Actors)	20
3.2.2 Yêu cầu của giảng viên	20
3.2.3 Yêu cầu của sinh viên.....	21
3.2.4 Yêu cầu của cố vấn.....	22
3.2.5 Yêu cầu của quản trị viên (Admin)	23
3.3. Lựa chọn công nghệ	24
3.3.1 Phía người dùng	24
3.3.2 Phía máy chủ	24
3.3.3 Tích hợp & dịch vụ ngoài	25
3.4. Thiết kế kiến trúc hệ thống tổng thể.....	25
3.5. Thiết kế cơ sở dữ liệu	35
3.6. Triển khai Chatbot.....	43
3.6.1 Mục tiêu triển khai chatbot	43
3.6.2 Kiến trúc triển khai Chatbot.....	44
CHƯƠNG 4. CÀI ĐẶT HỆ THỐNG VÀ KẾT QUẢ THỰC NGHIỆM	48

4.1. Xây dựng các API chức năng chính.....	48
4.2. Giao diện hệ thống	62
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	67
5.1. Kết luận	67
5.2. Hướng phát triển trong tương lai	67
DANH MỤC TÀI LIỆU THAM KHẢO.....	69

DANH MỤC HÌNH

Hình 3.1 Kiến trúc toàn bộ hệ thống	25
Hình 3.2 Luồng đồng bộ dữ liệu từ moodle	26
Hình 3.3 Luồng tính toán điểm tổng hợp của sinh viên.....	27
Hình 3.4 Luồng phát hiện và tạo cảnh báo rủi ro.....	29
Hình 3.5 Luồng chatbot tư vấn và trả lời câu hỏi	31
Hình 3.6 Luồng quản lý lịch sử chat	32
Hình 3.7 Luồng tra cứu và báo cáo	33
Hình 3.8 Lược đồ cơ sở dữ liệu	35
Hình 4.1 Giao diện trang đăng nhập	62
Hình 4.2 Giao diện trang chủ	62
Hình 4.3 Giao diện Chatbot	63
Hình 4.4 Giao diện chatbot hỗ trợ.....	64
Hình 4.5 Giao diện gợi ý nhanh	65
Hình 4.6 Giao diện Chatbot không sử dụng.....	66
Hình 4.7 Giao diện chatbot khi sử dụng	66

DANH MỤC BẢNG

Bảng 3.1 Bảng Actors	20
Bảng 3.2 Bảng yêu cầu của giảng viên	21
Bảng 3.3 Bảng yêu cầu của sinh viên	22
Bảng 3.4 Bảng yêu cầu của cố vấn	22
Bảng 3.5 Bảng yêu cầu của admin	23
Bảng 3.6 Mô tả bảng User.....	35
Bảng 3.7 Mô tả bảng Student.....	37
Bảng 3.8 Mô tả bảng Course.....	37
Bảng 3.9 Mô tả bảng grade_items.....	38
Bảng 3.10 Mô tả bảng Studen_grades	39
Bảng 3.11 Mô tả bảng Course_grade_sumaries.....	40
Bảng 3.12 Mô tả bảng enrollments	41
Bảng 3.13 Mô tả bảng Warnings	42
Bảng 3.14 Mô tả bảng Review ai	43
Bảng 4.1. Bảng mô tả API đồng bộ người dùng	Error! Bookmark not defined.
Bảng 4.2. Bảng mô tả API lấy danh sách sinh viên ..	Error! Bookmark not defined.
Bảng 4.3. Bảng mô tả API lấy danh sách giáo viên ..	Error! Bookmark not defined.
Bảng 4.4. Bảng mô tả API tìm sinh viên theo từ khóa.....	Error! Bookmark not defined.
Bảng 4.5 Bảng mô tả API lấy sinh viên theo mã sinh viên.....	Error! Bookmark not defined.
Bảng 4.6. Bảng mô tả API lấy người dùng theo Moodle ID...	Error! Bookmark not defined.
Bảng 4.7. Bảng mô tả API thống kê người dùng	Error! Bookmark not defined.

Bảng 4.8. Bảng mô tả API lấy danh sách sinh viên ..**Error! Bookmark not defined.**

Bảng 4.9. Bảng mô tả API lấy chi tiết sinh viên**Error! Bookmark not defined.**

Bảng 4.10. Bảng mô tả API phân tích rủi ro sinh viên **Error! Bookmark not defined.**

Bảng 4.11. Bảng mô tả API đồng bộ grade items**Error! Bookmark not defined.**

Bảng 4.12. Bảng mô tả API đồng bộ điểm sinh viên **Error! Bookmark not defined.**

Bảng 4.13. Bảng mô tả API lấy báo cáo điểm chi tiết **Error! Bookmark not defined.**

Bảng 4.14. Bảng mô tả API lấy toàn bộ điểm của sinh viên... **Error! Bookmark not defined.**

Bảng 4.15. Bảng mô tả API lấy tổng hợp điểm theo môn **Error! Bookmark not defined.**

Bảng 4.16. Bảng mô tả API lấy các đầu điểm của khóa học .. **Error! Bookmark not defined.**

Bảng 4.17. Bảng mô tả API lấy điểm trung bình sinh viên..... **Error! Bookmark not defined.**

Bảng 4.18. Bảng mô tả API lọc sinh viên hoàn thành thấp..... **Error! Bookmark not defined.**

Bảng 4.19. Bảng mô tả API xếp hạng sinh viên trong lớp..... **Error! Bookmark not defined.**

Bảng 4.20. Bảng mô tả API lấy danh sách cảnh báo **Error! Bookmark not defined.**

Bảng 4.21. Bảng mô tả API lấy cảnh báo mức độ**Error! Bookmark not defined.**

Bảng 4.22. Bảng mô tả API thống kê cảnh báo**Error! Bookmark not defined.**

Bảng 4.23. Bảng mô tả API xác nhận cảnh báo**Error! Bookmark not defined.**

Bảng 4.24. Bảng mô tả API tạo cảnh báo đồ thử nghiệm **Error! Bookmark not defined.**

Bảng 4.25. Bảng mô tả API chatbot gửi tin nhắn**Error! Bookmark not defined.**

Bảng 4.26. Bảng mô tả API lấy lịch sử chat**Error! Bookmark not defined.**

Bảng 4.27. Bảng mô tả API đồng bộ toàn bộ dữ liệu thủ công**Error! Bookmark not defined.**

Bảng 4.28. Bảng mô tả API kiểm tra kết nối Moodle..... **Error! Bookmark not defined.**

Bảng 4.29. Bảng mô tả API gửi thông báo thủ công **Error! Bookmark not defined.**

Bảng 4.30. Bảng mô tả API gửi thông báo hàng loạt**Error! Bookmark not defined.**

Bảng 4.31. Bảng mô tả API lấy thông báo của người dùng.... **Error! Bookmark not defined.**

Bảng 4.32. Bảng mô tả API lấy điểm sinh viên từ Moodle **Error! Bookmark not defined.**

Bảng 4.33. Bảng mô tả API lấy chuyên cần sinh viên **Error! Bookmark not defined.**

Bảng 4.34. Bảng mô tả API lấy trạng thái sinh viên.**Error! Bookmark not defined.**

Bảng 4.35. Bảng mô tả API lọc sinh viên nguy cơ ...**Error! Bookmark not defined.**

Bảng 4.36. Bảng mô tả API tổng quan lớp học.....**Error! Bookmark not defined.**

Bảng 4.37. Bảng mô tả API lấy lần truy cập cuối.....**Error! Bookmark not defined.**

Bảng 4.38. Bảng mô tả API lấy danh sách khóa học **Error! Bookmark not defined.**

Bảng 4.39. Bảng mô tả API gửi thông báo qua Moodle **Error! Bookmark not defined.**

Bảng 4.40. Bảng mô tả API lấy sinh viên chưa nộp bài **Error! Bookmark not defined.**

Bảng 4.40. Bảng mô tả API lọc sinh viên rủi ro theo môn **Error! Bookmark not defined.**

Bảng 4.40. Bảng mô tả API lấy sinh viên không hoạt động ... **Error! Bookmark not defined.**

Bảng 4.40. Bảng mô tả API tổng hợp rủi ro theo lớp**Error! Bookmark not defined.**

CHƯƠNG 1. MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại các cơ sở giáo dục đại học, hệ thống quản lý học tập (Learning Management System - LMS) ngày càng đóng vai trò quan trọng trong việc tổ chức lớp học, quản lý nội dung, theo dõi quá trình học tập và hỗ trợ đánh giá người học. Trong số đó, Moodle là một trong những hệ thống quản lý học tập mã nguồn mở phổ biến, cho phép giảng viên theo dõi điểm số, lịch sử truy cập, tình trạng hoàn thành bài tập và mức độ tham gia của sinh viên. Tuy nhiên, mặc dù các dữ liệu này đã sẵn có trong hệ thống, việc khai thác chúng một cách trực quan, nhanh chóng và có khả năng cảnh báo sớm vẫn còn nhiều hạn chế.

Trong thực tế, giảng viên thường phải quản lý nhiều lớp học, nhiều nhóm sinh viên và nhiều loại dữ liệu học tập khác nhau. Việc theo dõi thủ công từng sinh viên để phát hiện các dấu hiệu sa sút như điểm thấp, chuyên cần kém, không truy cập hệ thống trong thời gian dài hoặc không hoàn thành bài tập thường tốn rất nhiều thời gian và công sức. Điều này đặc biệt khó khăn trong các lớp có quy mô lớn, nơi số lượng sinh viên đông và tần suất cập nhật dữ liệu liên tục. Nếu không có công cụ hỗ trợ phù hợp, giảng viên có thể bỏ lỡ các trường hợp có nguy cơ cao, dẫn đến việc can thiệp không kịp thời.

Bên cạnh đó, việc phát hiện sớm sinh viên có nguy cơ thôi học là một nhiệm vụ rất quan trọng trong quản lý giáo dục đại học. Những sinh viên có biểu hiện giảm sút trong học tập thường cần được hỗ trợ, tư vấn hoặc nhắc nhở sớm để tránh tình trạng bỏ học giữa chừng. Nếu được phát hiện kịp thời, nhà trường và giảng viên có thể triển khai các biện pháp phù hợp nhằm cải thiện kết quả học tập và tăng tỷ lệ duy trì học tập của sinh viên. Vì vậy, một hệ thống có khả năng phân tích dữ liệu học tập và tự động đưa ra cảnh báo sẽ mang lại giá trị thực tiễn rất lớn.

Xuất phát từ nhu cầu đó, đề tài này được thực hiện nhằm xây dựng một hệ thống chatbot có khả năng hỗ trợ giảng viên trong việc tra cứu thông tin học tập, theo dõi tiến độ sinh viên và cảnh báo những trường hợp có nguy cơ thôi học. Hệ thống

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

được thiết kế để kết hợp giữa xử lý ngôn ngữ tự nhiên, khai thác dữ liệu từ Moodle và cơ chế phân tích nguy cơ học tập, từ đó tạo ra một công cụ hỗ trợ thông minh, tiện lợi và phù hợp với môi trường giáo dục hiện đại.

Ngoài ra, chatbot còn giúp đơn giản hóa quá trình tương tác giữa người dùng và hệ thống. Thay vì phải truy cập nhiều màn hình chức năng khác nhau, giảng viên chỉ cần đặt câu hỏi bằng ngôn ngữ tự nhiên để nhận được câu trả lời về điểm số, chuyên cần, tình trạng học tập hoặc danh sách sinh viên có nguy cơ. Điều này giúp nâng cao trải nghiệm người dùng, giảm thao tác thủ công và tăng tốc độ tra cứu thông tin. Với những lợi ích trên, việc lựa chọn đề tài này là hoàn toàn phù hợp cả về mặt thực tiễn lẫn giá trị nghiên cứu.

1.2. Mục tiêu

Mục tiêu của đề tài là xây dựng một hệ thống chatbot hỗ trợ giảng viên trong việc theo dõi tiến độ học tập của sinh viên và cảnh báo sớm các trường hợp có nguy cơ thôi học. Hệ thống phải đảm bảo khả năng tiếp nhận câu hỏi bằng ngôn ngữ tự nhiên, phân tích ý định người dùng, truy vấn dữ liệu học tập từ Moodle và tạo ra phản hồi dễ hiểu, chính xác, kịp thời.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Xây dựng hệ thống chatbot có khả năng tiếp nhận và xử lý câu hỏi bằng tiếng Việt trong môi trường giáo dục.
- Kết nối hệ thống với Moodle để lấy dữ liệu về khóa học, sinh viên, điểm số, chuyên cần và lịch sử học tập.
- Phân tích dữ liệu học tập nhằm xác định sinh viên có dấu hiệu sa sút hoặc có nguy cơ thôi học.
- Thiết kế cơ chế cảnh báo sớm cho giảng viên khi phát hiện sinh viên thuộc nhóm rủi ro.
- Cung cấp giao diện thân thiện, trực quan, dễ sử dụng cho giảng viên và sinh viên.
- Tổ chức lưu trữ dữ liệu và lịch sử hội thoại để phục vụ thống kê, tra cứu và mở rộng trong tương lai.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Ngoài các mục tiêu về chức năng, đề tài còn hướng tới một số mục tiêu phi chức năng như:
- Đảm bảo hệ thống hoạt động ổn định và phản hồi nhanh.
- Có kiến trúc rõ ràng, dễ bảo trì và mở rộng.
- Tăng khả năng tự động hóa trong việc giám sát học tập.
- Hạn chế tối đa thao tác thủ công của giảng viên.
- Đảm bảo tính minh bạch và khách quan trong công tác theo dõi, đánh giá sinh viên.

1.3. Nội dung nghiên cứu

Để đạt được các mục tiêu trên, đề tài tập trung nghiên cứu và triển khai các nội dung chính sau:

- Nghiên cứu cơ sở lý thuyết về hệ thống quản lý học tập Moodle, cơ chế Web Services và dữ liệu học tập có thể khai thác được từ hệ thống này.
- Tìm hiểu tổng quan về chatbot, xử lý ngôn ngữ tự nhiên và các mô hình nhận diện ý định người dùng trong môi trường hội thoại.
- Nghiên cứu các phương pháp đánh giá nguy cơ học tập dựa trên các chỉ số như điểm số, chuyên cần, mức độ hoạt động và thời gian truy cập gần nhất.
- Phân tích yêu cầu hệ thống từ góc độ người dùng là giảng viên, sinh viên và quản trị viên.
- Thiết kế kiến trúc tổng thể của hệ thống chatbot, bao gồm giao diện người dùng, tầng xử lý nghiệp vụ, tầng tích hợp dữ liệu và tầng cơ sở dữ liệu.
- Xây dựng các chức năng chính như đăng nhập, tra cứu điểm, kiểm tra chuyên cần, xem tình trạng học tập, cảnh báo sinh viên nguy cơ và lưu lịch sử hội thoại.
- Tích hợp cơ chế đồng bộ dữ liệu từ Moodle để đảm bảo dữ liệu luôn được cập nhật.
- Kiểm thử và đánh giá hệ thống dựa trên các tình huống sử dụng thực tế.

1.4. Đối tượng và phạm vi nghiên

Đối tượng nghiên cứu:

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Hệ thống quản lý học tập Moodle và các dữ liệu học tập có thể khai thác từ hệ thống này.
- Sinh viên đang tham gia các khóa học trên Moodle.
- Giảng viên sử dụng hệ thống để theo dõi và đánh giá tiến độ học tập.
- Các chỉ số phản ánh nguy cơ thôi học như điểm trung bình, chuyên cần, số ngày không truy cập, mức độ hoàn thành bài tập và lịch sử tương tác.

Phạm vi nghiên cứu:

- Hệ thống tập trung vào khai thác dữ liệu học tập từ Moodle, không đi sâu vào các hệ thống LMS khác.
- Đối tượng cảnh báo là sinh viên có nguy cơ học tập sa sút hoặc thôi học dựa trên dữ liệu hiện có.
- Chatbot chủ yếu hỗ trợ các truy vấn liên quan đến tình trạng học tập, điểm số, chuyên cần và cảnh báo rủi ro.
- Hệ thống không đi sâu vào chấm điểm tự động hay đánh giá năng lực chuyên môn toàn diện của sinh viên.
- Các chức năng AI chỉ mang tính hỗ trợ phân tích và tạo phản hồi, không thay thế hoàn toàn vai trò của giảng viên.
- Việc giới hạn phạm vi nghiên cứu giúp đề tài tập trung vào bài toán cốt lõi, đảm bảo tính khả thi trong thời gian thực hiện, đồng thời tạo nền tảng để mở rộng trong các giai đoạn tiếp theo.

1.5. Phương pháp nghiên cứu

Nghiên cứu lý thuyết:

- Phần nghiên cứu lý thuyết tập trung vào các nội dung sau:
- Tìm hiểu hệ thống Moodle, cơ chế Web Services và cách khai thác dữ liệu học tập.
- Nghiên cứu chatbot, xử lý ngôn ngữ tự nhiên và mô hình nhận diện ý định người dùng.
- Tìm hiểu các phương pháp đánh giá nguy cơ học tập và các chỉ số cảnh báo sớm.
- Nghiên cứu kiến trúc hệ thống web hiện đại, cơ sở dữ liệu quan hệ, cơ chế xác thực và phân quyền.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Nghiên cứu thực nghiệm:

- Phân nghiên cứu thực nghiệm tập trung vào:
- Phân tích yêu cầu thực tế của hệ thống.
- Thiết kế và xây dựng các chức năng của chatbot.
- Tích hợp hệ thống với Moodle để lấy dữ liệu học tập.
- Thiết kế cơ chế phân tích và cảnh báo nguy cơ.
- Kiểm thử các tình huống sử dụng thực tế.
- Đánh giá mức độ phù hợp của hệ thống với nhu cầu giảng dạy và quản lý học tập.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống quản lý học tập Moodle

Moodle (Modular Object-Oriented Dynamic Learning Environment) là một hệ thống quản lý học tập mã nguồn mở, được phát triển bởi Martin Dougiamas vào năm 2002. Ban đầu, Moodle được thiết kế nhằm hỗ trợ giáo dục trực tuyến dựa trên triết lý học tập kiến tạo, khuyến khích người học chủ động tham gia, tương tác và xây dựng tri thức. Qua nhiều năm phát triển, Moodle đã trở thành một trong những nền tảng LMS phổ biến nhất thế giới, được sử dụng rộng rãi tại các trường đại học, cơ sở đào tạo và tổ chức doanh nghiệp ở hơn 200 quốc gia.

2.1.1 Kiến trúc Moodle

Moodle cung cấp nhiều thành phần dữ liệu có giá trị cho quá trình theo dõi học tập, bao gồm thông tin khóa học, danh sách sinh viên, bảng điểm, lịch sử nộp bài, trạng thái hoàn thành hoạt động và thời gian truy cập gần nhất. Những dữ liệu này giúp giảng viên có cái nhìn toàn diện hơn về quá trình học tập của từng sinh viên, từ đó dễ dàng phát hiện các trường hợp cần chú ý. [1]

Thay vì phải kiểm tra thủ công từng lớp học hoặc từng sinh viên, hệ thống có thể tổng hợp và phân tích dữ liệu tự động để xác định các trường hợp có dấu hiệu bất thường. Điều này đặc biệt hữu ích trong các lớp học có số lượng sinh viên lớn, nơi việc theo dõi bằng phương pháp truyền thống thường tốn nhiều thời gian và dễ bỏ sót thông tin quan trọng.

2.1.2 Moodle Web Services

Moodle hỗ trợ Web Services để cho phép các hệ thống bên ngoài truy cập và thao tác dữ liệu một cách có kiểm soát. Cơ chế này sử dụng token xác thực và cho phép thực hiện các chức năng như truy vấn người dùng, khóa học, điểm số, danh sách đăng ký và nhiều dịch vụ khác. Đây là nền tảng quan trọng để tích hợp Moodle với các ứng dụng bên ngoài. [2]

Việc sử dụng Web Services trong đề tài này có ý nghĩa quan trọng vì giúp hệ thống chatbot truy cập dữ liệu Moodle an toàn, giảm phụ thuộc vào thao tác thủ công, tăng khả năng đồng bộ hóa dữ liệu và hỗ trợ triển khai các chức năng cảnh báo theo thời

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
gian gần thực. Nhờ vậy, hệ thống có thể phản hồi nhanh hơn và chính xác hơn đối với các yêu cầu của giảng viên và sinh viên.

2.2. Khái niệm chatbot và ứng dụng trong giáo dục

Chatbot là một hệ thống phần mềm có khả năng giao tiếp với con người thông qua ngôn ngữ tự nhiên, thường dưới dạng văn bản. Chatbot được thiết kế nhằm trả lời câu hỏi, hỗ trợ tra cứu thông tin, tự động hóa giao tiếp và giảm tải công việc cho con người trong các tác vụ lặp lại. Trong những năm gần đây, chatbot đã được ứng dụng rộng rãi trong nhiều lĩnh vực như thương mại điện tử, chăm sóc khách hàng, y tế và giáo dục. [3]

Trong giáo dục, chatbot mang lại nhiều lợi ích thiết thực như hỗ trợ sinh viên tra cứu thông tin nhanh chóng, giúp giảng viên tiếp cận dữ liệu học tập thuận tiện hơn, tự động trả lời các câu hỏi thường gặp, cung cấp phản hồi cá nhân hóa và tăng tính tương tác giữa người dùng với hệ thống. Với đặc thù của môi trường học tập hiện đại, chatbot đang dần trở thành một công cụ hỗ trợ hiệu quả trong quản lý và giảng dạy.

Với đề tài này, chatbot được định hướng như một công cụ hỗ trợ giảng viên theo dõi tình trạng học tập của sinh viên. Hệ thống không chỉ trả lời câu hỏi dạng tra cứu thông tin mà còn có khả năng phân tích dữ liệu để đưa ra cảnh báo về nguy cơ học tập, từ đó hỗ trợ giảng viên can thiệp kịp thời. Cách xây dựng này giúp chatbot mang tính thực tiễn cao và phù hợp với nhu cầu quản lý học tập trong môi trường đại học.

2.2.1 Phân loại chatbot

Có thể phân chatbot thành ba nhóm chính gồm chatbot theo luật, chatbot dựa trên trí tuệ nhân tạo và chatbot lai. Chatbot theo luật hoạt động dựa trên các quy tắc hoặc mẫu câu được xây dựng sẵn; chatbot dựa trên trí tuệ nhân tạo sử dụng mô hình học máy hoặc mô hình ngôn ngữ để hiểu ngữ cảnh và phản hồi linh hoạt; chatbot lai kết hợp cả hai phương pháp nhằm đảm bảo vừa chính xác vừa linh hoạt. [3]

Trong đề tài này, hệ thống sử dụng mô hình lai. Một mặt, hệ thống dùng luật nhận diện ý định để xử lý các câu hỏi đơn giản và thường gặp; mặt khác, có thể mở rộng bằng mô hình AI để xử lý những câu hỏi phức tạp hơn. Cách tiếp cận này giúp hệ thống vừa tiết kiệm tài nguyên, vừa nâng cao khả năng hiểu ngôn ngữ tự nhiên của người dùng.

2.2.2 Vai trò của chatbot trong cảnh báo sớm

Chatbot hỗ trợ cảnh báo sớm có thể giúp giảng viên phát hiện sinh viên có điểm số thấp kéo dài, chuyên cần giảm, không truy cập hệ thống trong thời gian dài, không hoàn thành bài tập hoặc có dấu hiệu giảm sút tương tác học tập. Từ đó, giảng viên có thể đưa ra biện pháp can thiệp sớm như nhắc nhở, tư vấn hoặc theo dõi sát hơn.

Đây là điểm khác biệt quan trọng của chatbot trong đề tài này so với các chatbot thông thường chỉ mang tính hỏi đáp đơn giản. Việc kết hợp giữa giao tiếp ngôn ngữ tự nhiên và phân tích dữ liệu học tập giúp hệ thống không chỉ hỗ trợ tra cứu mà còn hỗ trợ ra quyết định trong công tác quản lý học tập. [3]

2.3. Xử lý ngôn ngữ tự nhiên trong chatbot

Xử lý ngôn ngữ tự nhiên là lĩnh vực của trí tuệ nhân tạo nghiên cứu cách giúp máy tính hiểu, phân tích và sinh ra ngôn ngữ của con người. Trong hệ thống chatbot, NLP giữ vai trò trung tâm vì nó quyết định khả năng hiểu ý định người dùng và phản hồi chính xác. [3]

Một hệ thống chatbot sử dụng NLP thường trải qua các bước gồm tiếp nhận câu hỏi của người dùng, chuẩn hóa và tiền xử lý văn bản, nhận diện từ khóa và thực thể, phân loại ý định, rồi tạo phản hồi phù hợp. Quá trình này cho phép hệ thống chuyển các câu hỏi tự nhiên thành những yêu cầu có cấu trúc để xử lý tự động.

Trong hệ thống này, người dùng có thể đặt câu hỏi theo nhiều cách khác nhau, ví dụ “Sinh viên Nguyễn Văn A có bị cảnh báo không?”, “Cho tôi xem danh sách sinh viên có nguy cơ cao”, hoặc “Điểm của em hiện tại thế nào?”. Mặc dù cùng một mục đích, các câu hỏi có thể có cấu trúc ngôn ngữ rất khác nhau, do đó chatbot cần có khả năng hiểu ngữ nghĩa chứ không chỉ dựa vào từ khóa cố định.

2.3.1 Nhận diện ý định

Nhận diện ý định là bước xác định mục đích chính của câu hỏi. Trong hệ thống, đây là bước quan trọng để phân loại câu hỏi của người dùng thành các nhóm như xem điểm, xem chuyên cần, kiểm tra trạng thái học tập, xem danh sách sinh viên nguy cơ, xem sinh viên ngừng tương tác hay cấu hình ngưỡng cảnh báo. [3]

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Trong thực tế, việc nhận diện đúng ý định giúp hệ thống phản hồi nhanh với các câu hỏi phổ biến mà không cần gọi mô hình AI ở mọi trường hợp. Điều này vừa tiết kiệm tài nguyên, vừa tăng tốc độ phản hồi, đồng thời đảm bảo hệ thống có thể vận hành ổn định trong điều kiện sử dụng thực tế.

2.3.2 Trích xuất thực tế

Bên cạnh intent, hệ thống còn cần trích xuất các thực thể quan trọng như mã số sinh viên, tên môn học, tên lớp, mức cảnh báo và số ngày không truy cập. Đây là những thông tin quyết định đến việc truy vấn dữ liệu chính xác trong cơ sở dữ liệu hoặc từ Moodle. [2]

Ví dụ, trong câu “Sinh viên 110122223 có bị cảnh báo không?”, thực thể “110122223” là dữ liệu cốt lõi để truy vấn đúng hồ sơ sinh viên. Việc nhận diện thực thể giúp chatbot hiểu rõ hơn ngữ cảnh và tạo phản hồi chính xác hơn, đặc biệt trong các truy vấn mang tính cá nhân hóa cao.

2.4. Trí tuệ nhân tạo và mô hình ngôn ngữ lớn trong chatbot

Trí tuệ nhân tạo là lĩnh vực nghiên cứu nhằm xây dựng các hệ thống có khả năng thực hiện những nhiệm vụ vốn cần trí tuệ của con người như suy luận, học hỏi, nhận diện ngôn ngữ, phân loại dữ liệu và sinh phản hồi. Trong các hệ thống hội thoại hiện đại, AI là nền tảng giúp chatbot hiểu và phản ứng linh hoạt hơn với câu hỏi của người dùng. [3]

Mô hình ngôn ngữ lớn là một nhánh quan trọng của AI, được huấn luyện trên tập dữ liệu rất lớn để có thể hiểu và sinh ngôn ngữ tự nhiên một cách linh hoạt. Theo các nghiên cứu gần đây, kiến trúc Transformer với cơ chế self-attention là nền tảng quan trọng giúp mô hình hiểu mối quan hệ giữa các từ trong câu và giữa các câu trong ngữ cảnh rộng. Đây là lý do các mô hình ngôn ngữ lớn ngày càng được ứng dụng mạnh mẽ trong các hệ thống hội thoại thông minh.

2.4.1 Ứng dụng AI trong phân tích học tập

Trong đề tài này, AI được ứng dụng để phân tích dữ liệu học tập, diễn giải trạng thái sinh viên, đưa ra câu trả lời tự nhiên hơn cho giảng viên và hỗ trợ phân tích các trường hợp nguy cơ mà luật cứng khó bao phủ đầy đủ. AI không thay thế hoàn toàn các

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
quy tắc nghiệp vụ mà đóng vai trò hỗ trợ nhằm nâng cao độ linh hoạt và khả năng mở rộng của hệ thống.

Ví dụ, nếu giảng viên hỏi “Những sinh viên nào cần theo dõi đặc biệt trong lớp Cơ sở dữ liệu?”, chatbot không chỉ cần liệt kê tên sinh viên mà còn phải hiểu đây là yêu cầu lọc nhóm sinh viên theo mức nguy cơ, sau đó tổng hợp và trình bày kết quả ngắn gọn, rõ ràng. Năng lực này là ưu điểm nổi bật của việc ứng dụng AI trong xử lý ngôn ngữ tự nhiên. [3]

2.4.2 Kết hợp luật và AI

Trong thực tế, một chatbot giáo dục hiệu quả thường không phụ thuộc hoàn toàn vào AI hoặc hoàn toàn vào luật cứng. Nếu dùng AI cho mọi câu hỏi, hệ thống có thể chậm hơn và khó kiểm soát; ngược lại, nếu chỉ dùng luật cứng, hệ thống sẽ hạn chế về khả năng hiểu ngôn ngữ tự nhiên. [3]

Do đó, hướng tiếp cận phù hợp là dùng luật cục bộ để xử lý các câu hỏi đơn giản, phổ biến; dùng AI cho các câu hỏi phức tạp hoặc mơ hồ; và kết hợp cả hai để đạt hiệu quả cao nhất. Cách thiết kế này phù hợp với hệ thống trong đề tài, đồng thời giúp tăng khả năng mở rộng trong tương lai.

2.5. Cơ sở lý thuyết về phát hiện sinh viên có nguy cơ thôi học

Phát hiện sinh viên có nguy cơ thôi học là quá trình thu thập và phân tích dữ liệu học tập để xác định những sinh viên có dấu hiệu sa sút, từ đó đưa ra cảnh báo và hỗ trợ kịp thời. Đây là một bài toán quan trọng trong giáo dục đại học vì việc phát hiện sớm có thể góp phần giảm tỷ lệ bỏ học và nâng cao chất lượng đào tạo. [1]

Các dấu hiệu thường được xem xét gồm điểm trung bình thấp, chuyên cần giảm, không truy cập hệ thống trong thời gian dài, không hoàn thành bài tập hoặc hoạt động học tập và ít tương tác với hệ thống. Những chỉ số này phản ánh mức độ gắn kết của sinh viên với quá trình học tập và có thể dùng để xây dựng các tiêu chí cảnh báo sớm.

2.5.1 Các chỉ số đánh giá nguy cơ

Trong hệ thống, một sinh viên có thể được đánh giá dựa trên các chỉ số như điểm trung bình học tập, tỷ lệ chuyên cần, số ngày không truy cập, tỷ lệ hoàn thành bài tập và

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
số lượng môn có dấu hiệu bất thường. Đây là các chỉ số có tính thực tiễn cao vì đều có thể thu thập từ hệ thống quản lý học tập. [1]

Từ các chỉ số này, hệ thống có thể phân loại sinh viên thành ba mức: xanh, vàng và đỏ. Mức xanh thể hiện trạng thái an toàn; mức vàng cho thấy cần theo dõi; mức đỏ là nguy cơ cao và cần can thiệp ngay. Phân loại theo mức rủi ro giúp giảng viên dễ dàng xác định nhóm sinh viên cần được hỗ trợ trước tiên.

2.5.2 Phân loại công cụ đánh giá mã nguồn

Các công cụ đánh giá mã nguồn có thể được phân loại thành ba nhóm chính, mỗi nhóm có một phương thức hoạt động và mục tiêu riêng:

- *Phân tích tĩnh (Static Code Analysis)*: là phương pháp phân tích mã nguồn mà không cần thực thi chương trình. Nó dựa vào cú pháp, cấu trúc mã và một bộ quy tắc lập trình đã định sẵn để tìm kiếm các vấn đề. Quá trình này diễn ra một cách tự động và thường là một phần của quy trình CI/CD. Các vấn đề thường phát hiện như: lỗi cú pháp, vấn đề về coding style (quy chuẩn mã hóa), lỗ hổng bảo mật tiềm ẩn, mã dư thừa hoặc phức tạp quá mức...

Ví dụ: SonarQube, ESLint (JavaScript), Checkstyle (Java), Pylint (Python),...

- *Phân tích động (Dynamic Code Analysis)*: là phương pháp phân tích mã nguồn bằng cách thực thi chương trình trong một môi trường giả lập hoặc môi trường thực tế. Quá trình này giúp phát hiện các vấn đề chỉ xuất hiện trong quá trình chạy (runtime), đặc biệt là các vấn đề liên quan đến hiệu suất và tương tác với hệ thống.

Ví dụ: Valgrind, JProfiler, AppDynamics.

- *Phân tích hỗn hợp (Hybrid Analysis)*: là sự kết hợp giữa cả hai phương pháp phân tích tĩnh và động để tăng độ chính xác và phạm vi phát hiện. Công cụ phân tích hỗn hợp thường sử dụng kết quả từ phân tích tĩnh để hướng dẫn quá trình phân tích động, từ đó tìm kiếm các lỗ hổng bảo mật phức tạp hoặc các lỗi logic tinh vi.

2.5.3 Quy trình đánh giá mã nguồn

Một công cụ đánh giá mã nguồn hoạt động như một hệ thống tự động hóa, tuân theo một quy trình chặt chẽ để phân tích và cung cấp phản hồi về chất lượng mã. Quy trình này thường bao gồm các giai đoạn sau:

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- *Giai đoạn thu thập mã nguồn*: Đây là bước khởi đầu, nơi công cụ tiếp cận và lấy mã nguồn cần phân tích. Quy trình này có thể thực hiện theo hai cách chính:

+ Truy cập cục bộ (Local Access): Công cụ trực tiếp đọc và phân tích mã nguồn từ một thư mục cụ thể trên máy tính hoặc máy chủ.

+ Tích hợp với hệ thống quản lý mã nguồn (VCS Integration): Đây là phương pháp phổ biến nhất trong môi trường làm việc nhóm hiện đại. Công cụ sẽ kết nối với các hệ thống quản lý mã nguồn như GitHub, GitLab hoặc Bitbucket thông qua các API hoặc plugin. Điều này cho phép công cụ tự động lấy mã nguồn mới nhất mỗi khi có một commit hoặc pull request. Việc tích hợp này đảm bảo rằng mọi thay đổi đều được kiểm tra một cách nhất quán và liên tục.

- *Giai đoạn phân tích cú pháp (Parsing)*: Sau khi thu thập mã nguồn, công cụ sẽ tiến hành phân tích cú pháp để hiểu cấu trúc của mã. Thay vì đọc mã dưới dạng văn bản thuần, công cụ sẽ chuyển đổi mã nguồn thành một cấu trúc dữ liệu gọi là Cây Cú pháp Trừu tượng (Abstract Syntax Tree - AST).

+ Cấu trúc của AST: AST biểu diễn cấu trúc ngữ pháp của mã nguồn dưới dạng một cây, trong đó các node của cây là các cấu trúc lập trình như hàm, biến, vòng lặp, biểu thức, ...

+ Lợi ích của AST: Việc chuyển đổi sang AST giúp công cụ dễ dàng thực hiện các thao tác phân tích phức tạp. Nó có thể duyệt qua cấu trúc cây để kiểm tra logic, tìm kiếm các mẫu mã không phù hợp, hoặc xác định các mối quan hệ giữa các thành phần khác nhau của mã nguồn một cách hiệu quả hơn nhiều so với việc phân tích chuỗi ký tự.

- *Giai đoạn áp dụng quy tắc kiểm tra*: Đây là giai đoạn cốt lõi của quá trình phân tích. Công cụ sẽ so sánh AST của mã nguồn với một bộ quy tắc kiểm tra (ruleset) đã được định sẵn. Bộ quy tắc này được thiết kế để tìm kiếm các vấn đề cụ thể, bao gồm:

+ Quy chuẩn mã hóa (Coding Standards): Các quy tắc về định dạng mã, đặt tên biến, cấu trúc hàm... Ví dụ: PEP8 cho Python, Google Style Guide cho Java.

+ Lỗi và Bug: Các quy tắc để tìm lỗi logic tiềm ẩn, ví dụ như một biến được khai báo nhưng không bao giờ được sử dụng, hoặc một đoạn mã không thể truy cập được.

+ Lỗ hổng bảo mật: Các quy tắc để phát hiện các lỗ hổng phổ biến như SQL injection, XSS, hoặc việc sử dụng các hàm không an toàn.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

+ Vấn đề hiệu suất: Các quy tắc để xác định những đoạn mã có thể gây ra hiệu suất kém, chẳng hạn như các vòng lặp lồng nhau quá sâu.

- *Giai đoạn phát hiện và báo cáo*: Khi các quy tắc được áp dụng, công cụ sẽ ghi lại tất cả các vấn đề được tìm thấy. Kết quả này sau đó được tổng hợp và trình bày dưới dạng một báo cáo chi tiết. Báo cáo này thường bao gồm:

+ Thông tin chi tiết về vấn đề: Tên lỗi, mô tả, vị trí chính xác (số dòng) trong mã nguồn.

+ Mức độ nghiêm trọng: Các vấn đề được phân loại theo mức độ nghiêm trọng (ví dụ: Minor, Major, Critical, Blocker).

+ Đề xuất sửa đổi (Fix Suggestion): Nhiều công cụ hiện đại còn cung cấp các gợi ý cụ thể về cách khắc phục vấn đề.

- *Giai đoạn tích hợp vào quy trình CI/CD*: Để tối đa hóa hiệu quả, các công cụ đánh giá mã nguồn được tích hợp chặt chẽ vào quy trình CI/CD.

+ Kích hoạt tự động: Công cụ sẽ tự động chạy mỗi khi có một sự kiện quan trọng xảy ra, chẳng hạn như lập trình viên push mã mới hoặc tạo một pull request.

+ Điều kiện chấp nhận (Quality Gate): Hệ thống CI/CD có thể được cấu hình để kiểm tra kết quả báo cáo. Nếu công cụ phát hiện lỗi nghiêm trọng (ví dụ: một lỗi blocker), quy trình build có thể bị dừng lại và ngăn mã lỗi được hợp nhất vào nhánh chính.

+ Phản hồi tức thì: Lập trình viên sẽ nhận được phản hồi ngay lập tức về chất lượng mã của mình, giúp họ sửa lỗi kịp thời trước khi mã được tích hợp sâu hơn vào dự án.

2.5.4 Kết hợp công cụ đánh giá mã nguồn với GitHub Actions

Việc tích hợp công cụ đánh giá mã nguồn với GitHub Actions là một phương pháp luận cốt lõi trong quy trình CI/CD hiện đại, nhằm tự động hóa toàn bộ quá trình kiểm tra chất lượng mã nguồn ngay khi có thay đổi trong kho lưu trữ. Sự kết hợp này tạo ra một "cổng chất lượng" (quality gate) tự động, giúp phát hiện và xử lý lỗi một cách chủ động, ngăn chặn mã kém chất lượng được hợp nhất vào nhánh chính của dự án.

Một quy trình tích hợp giữa công cụ đánh giá mã nguồn và GitHub Actions thường bao gồm các giai đoạn sau:

- *Cấu hình workflow github actions*: Quy trình bắt đầu bằng việc tạo một tệp tin cấu hình YAML trong thư mục `.github/workflows` của kho mã nguồn. Tệp tin này định

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

nghĩa các công việc (jobs) và các bước (steps) cần thực hiện. Workflow được cấu hình để tự động kích hoạt bởi các sự kiện trên GitHub. Phổ biến nhất là pull_request (khi một pull request được mở hoặc cập nhật) hoặc push (khi mã nguồn được đẩy lên một nhánh cụ thể). Sự kiện này đảm bảo rằng mọi thay đổi đều được kiểm tra.

- *Thiết lập và thực thi phân tích*: Trong các bước của workflow, công cụ đánh giá mã nguồn (ví dụ: SonarQube Scanner, ESLint, Pylint) sẽ được cài đặt và cấu hình. Quá trình này có thể thực hiện thông qua các lệnh shell hoặc sử dụng các action có sẵn trên GitHub Marketplace. Sau khi cài đặt, công cụ sẽ được chạy trên mã nguồn mới nhất. Đối với các công cụ phân tích tĩnh, quá trình này bao gồm việc phân tích cú pháp mã nguồn thành cây cú pháp trừu tượng (AST) và so sánh với bộ quy tắc đã định sẵn.

- *Sinh báo cáo và xử lý kết quả*: Kết quả phân tích sẽ được tổng hợp thành một báo cáo chi tiết. Đối với các công cụ như SonarQube, báo cáo này sẽ được gửi đến máy chủ SonarQube Server hoặc SonarCloud để hiển thị trên dashboard. Kết quả kiểm tra cũng được phản hồi trực tiếp trên giao diện của github pull request dưới dạng các “checks” (kiểm tra). Tùy thuộc vào kết quả, check có thể hiển thị trạng thái “passed” (đã qua) hoặc “failed” (thất bại). Đây là một cơ chế quan trọng. Dựa trên các ngưỡng chất lượng đã định, workflow có thể được cấu hình để tự động đánh dấu quá trình kiểm tra là thất bại. Điều này có thể được sử dụng để chặn quá trình hợp nhất mã, yêu cầu lập trình viên phải sửa lỗi trước khi mã được chấp nhận.

Lợi ích khi tích hợp:

- *Phản hồi tức thời và liên tục*: Lập trình viên nhận được kết quả phân tích chỉ vài phút sau khi đẩy mã, cho phép họ sửa lỗi kịp thời.

- *Đảm bảo chất lượng liên tục*: Tự động hóa việc kiểm tra mọi thay đổi, từ đó duy trì một tiêu chuẩn chất lượng nhất quán và cao cho toàn bộ dự án.

- *Nâng cao tính minh bạch*: Kết quả phân tích được hiển thị công khai trên GitHub, giúp mọi thành viên trong nhóm dễ dàng theo dõi và giám sát chất lượng mã.

- *Tiết kiệm thời gian và công sức*: Giảm thiểu đáng kể thời gian và công sức cần thiết cho việc kiểm tra mã thủ công và debug.

2.6. Các công nghệ sử dụng trong hệ thống

hệ thống được hiện thực bằng các công nghệ chính sau:

- Java / Spring Boot: nền tảng backend chính.
- MySQL: lưu trữ dữ liệu có cấu trúc.
- YAML: cấu hình ứng dụng.
- Moodle: nguồn dữ liệu học tập.
- AI service: hỗ trợ phân tích câu hỏi khi cần.
- Repository pattern / service layer: tách riêng logic nghiệp vụ và logic truy xuất dữ liệu.

2.6.1 Ngôn ngữ lập trình Java

Java là ngôn ngữ lập trình hướng đối tượng (OOP), đa nền tảng, được phát triển bởi Sun Microsystems (hiện thuộc Oracle). Với triết lý "Write Once, Run Anywhere" (Viết một lần, chạy mọi nơi), Java sử dụng máy ảo (JVM - Java Virtual Machine) để biên dịch mã nguồn, đảm bảo tính ổn định và bảo mật cao trong các hệ thống doanh nghiệp lớn.

Đề tài sử dụng Java 17, phiên bản Long-Term Support (LTS), giúp tận dụng các tính năng mới như *Pattern Matching*, *Records*, và *Text Blocks*, giúp mã nguồn trở nên gọn gàng, hiệu quả và tối ưu hóa hiệu năng cho các ứng dụng backend hiện đại.

2.6.2 Spring Boot Framework

2.6.2.1 Tổng quan về Spring Boot

Spring Boot là một dự án mở rộng từ Spring Framework, được thiết kế để đơn giản hóa quá trình phát triển các ứng dụng dựa trên nền tảng Java. Spring Boot tự động cấu hình (Auto-configuration) các thư viện phụ thuộc, loại bỏ sự phức tạp của việc cấu hình XML truyền thống, giúp lập trình viên tập trung vào logic nghiệp vụ (business logic).

2.6.2.2 Kiến trúc Spring Boot

Kiến trúc của Spring Boot dựa trên nguyên lý *Inversion of Control (IoC)* và *Dependency Injection (DI)*, giúp các thành phần trong hệ thống có tính độc lập cao, dễ dàng kiểm thử và bảo trì.

2.6.2.3 Ưu điểm trong ứng dụng Chatbot

Trong đề tài này, Spring Boot đóng vai trò là "xương sống" (backbone) cho hệ thống vì các lý do:

- Hỗ trợ RESTful API: Dễ dàng xây dựng các Endpoint để tương tác với chatbot và các hệ thống khác.
- Hệ sinh thái phong phú: Tích hợp sẵn các module như Spring Security (bảo mật), Spring Data (truy xuất dữ liệu), và Spring Web (WebSocket/HTTP).
- Khả năng mở rộng: Kiến trúc module hóa giúp dễ dàng tích hợp thêm các dịch vụ AI hoặc API từ bên thứ ba như Claude API.

2.6.3 MySQL

Quy trình kết nối GitHub và thiết lập Webhook là một thành phần quan trọng trong hệ thống theo dõi và đánh giá dự án phần mềm, nhằm bảo đảm việc đồng bộ dữ liệu giữa nền tảng quản lý mã nguồn và hệ thống giám sát được thực hiện liên tục và kịp thời.

Về mặt khái niệm, kết nối GitHub được thực hiện thông qua việc sử dụng Giao diện lập trình ứng dụng (API) của GitHub để truy cập vào thông tin kho mã nguồn, danh sách thành viên, lịch sử hoạt động và các sự kiện phát sinh trong quá trình phát triển phần mềm. Cơ chế này cho phép hệ thống thu thập dữ liệu một cách tự động, thay vì yêu cầu thao tác thủ công từ người dùng.

Sau khi thiết lập kết nối, hệ thống tiến hành cấu hình Webhook, một cơ chế thông báo theo sự kiện mà GitHub cung cấp. Webhook cho phép GitHub gửi thông tin trực tiếp đến hệ thống mỗi khi xảy ra một hành động cụ thể, chẳng hạn như việc đẩy mã nguồn (push), tạo yêu cầu hợp nhất (pull request), hoặc bình luận trong quá trình xem xét mã. Nhờ đó, hệ thống có thể phản ứng gần như tức thời với những thay đổi, cập nhật các báo cáo tiến độ, kích hoạt quá trình phân tích mã nguồn, hoặc gửi phản hồi tự động cho nhóm phát triển.

Việc kết hợp giữa API và Webhook mang lại hai lợi ích bổ sung: API đảm bảo khả năng truy xuất dữ liệu có hệ thống, phục vụ các báo cáo tổng hợp định kỳ; trong khi Webhook cung cấp luồng dữ liệu thời gian thực, hỗ trợ cơ chế giám sát liên tục. Sự kết hợp này giúp hệ thống duy trì tính chính xác và kịp thời của thông tin, đồng thời giảm thiểu độ trễ trong việc phản hồi và xử lý sự kiện.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Như vậy, quy trình kết nối GitHub và nhận Webhook đóng vai trò như “mạch máu dữ liệu” của toàn bộ hệ thống, bảo đảm rằng mọi hoạt động phát triển của sinh viên đều được ghi nhận, phân tích và phản hồi một cách có hệ thống và minh bạch.

2.6.4 Quy trình hiển thị thống kê đóng góp

Quy trình hiển thị thống kê đóng góp trong hệ thống nhằm cung cấp cái nhìn toàn diện và minh bạch về mức độ tham gia của từng thành viên trong nhóm dự án. Đây là một thành phần quan trọng hỗ trợ giảng viên trong công tác đánh giá, đồng thời giúp sinh viên tự nhận thức về hiệu suất làm việc của bản thân và nhóm.

Cơ chế vận hành bắt đầu từ việc thu thập dữ liệu hoạt động từ kho mã nguồn GitHub thông qua API và Webhook. Các sự kiện như commit, pull request, review, hoặc issue comment được ghi nhận và lưu trữ vào cơ sở dữ liệu của hệ thống. Thông tin này không chỉ bao gồm nội dung thay đổi, mà còn chứa siêu dữ liệu (metadata) như thời gian thực hiện, người thực hiện, và phạm vi ảnh hưởng của thay đổi.

Sau giai đoạn thu thập, hệ thống tiến hành xử lý và tổng hợp dữ liệu. Quá trình này có thể bao gồm phân loại hoạt động (ví dụ: đóng góp mã mới, sửa lỗi, viết tài liệu), tính toán các chỉ số định lượng như số dòng mã được bổ sung/xóa, số lượng commit hợp lệ, hoặc tần suất tham gia. Để đảm bảo công bằng, hệ thống có thể áp dụng các thuật toán loại bỏ nhiễu, chẳng hạn bỏ qua những thay đổi không đáng kể hoặc các commit tự động.

Kết quả được trình bày trên giao diện thống kê trực quan, dưới dạng biểu đồ cột, biểu đồ tròn hoặc đường thời gian, cho phép so sánh mức độ đóng góp giữa các thành viên và theo dõi xu hướng làm việc của nhóm theo thời gian. Ngoài ra, các chỉ số này có thể được tích hợp vào bảng đánh giá tổng hợp của dự án, giúp giảng viên đưa ra quyết định chấm điểm dựa trên cả chất lượng và khối lượng công việc.

Việc hiển thị thống kê đóng góp không chỉ phục vụ mục đích đánh giá, mà còn đóng vai trò như một công cụ điều phối nhóm. Sinh viên có thể quan sát mức độ tham gia của đồng đội, từ đó chủ động điều chỉnh phân công công việc, đảm bảo tiến độ và cân bằng khối lượng giữa các thành viên. Điều này góp phần hình thành môi trường làm việc minh bạch, thúc đẩy tinh thần trách nhiệm và hợp tác trong nhóm.

CHƯƠNG 3. HIỆN THỰC HÓA NGHIÊN CỨU

3.1. Mô tả bài toán thực tế

Trong bối cảnh giáo dục đại học hiện nay, công tác quản lý tiến độ học tập và hỗ trợ sinh viên là một thách thức không nhỏ, đặc biệt là việc phát hiện kịp thời các cá nhân có nguy cơ bỏ học. Các phương pháp truyền thống như sử dụng sổ tay, email, hay các bảng tính rời rạc không còn đáp ứng được yêu cầu cập nhật thông tin chính xác và kịp thời trong môi trường đào tạo hiện đại.

Đặc biệt, việc theo dõi chuyên cần, tiến độ hoàn thành các học phần và kết quả học tập của sinh viên đang gặp nhiều khó khăn. Giảng viên và các cố vấn học tập đối mặt với thách thức lớn khi phải giám sát hàng trăm sinh viên mà thiếu đi các công cụ tự động hóa. Việc cập nhật trạng thái học tập thủ công không chỉ dẫn đến sai sót, thông tin thiếu đồng bộ mà còn làm mất đi cơ hội hỗ trợ sinh viên kịp thời khi họ bắt đầu có dấu hiệu sa sút hoặc có nguy cơ thôi học.

Thêm vào đó, việc phân tích nguyên nhân sinh viên có nguy cơ bỏ học hiện nay chủ yếu dựa trên cảm tính hoặc báo cáo cuối kỳ, thiếu các tiêu chí đánh giá khách quan và định lượng dựa trên dữ liệu thực tế từ hệ thống quản lý đào tạo (LMS). Việc thiếu một cơ chế cảnh báo sớm dựa trên các ngưỡng cụ thể (như tỷ lệ hoàn thành, điểm trung bình, hoặc tần suất truy cập) khiến các nhà quản lý và giảng viên khó đưa ra các biện pháp can thiệp hiệu quả.

Các giải pháp quản lý hiện có mặc dù mạnh mẽ nhưng thường không được thiết kế chuyên biệt cho việc giám sát tiến độ học tập và cảnh báo nguy cơ thôi học trong môi trường giáo dục. Những công cụ này thiếu tính năng tích hợp sâu với hệ thống LMS của trường đại học, không có cơ chế phân tích hành vi học tập, và thường có giao diện phức tạp, không phù hợp với nhu cầu tra cứu thông tin nhanh chóng của sinh viên và giảng viên.

Từ thực trạng trên, nảy sinh nhu cầu cấp thiết về một hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học. Hệ thống này cần tập trung lưu trữ và quản lý dữ liệu từ LMS, phân tích chuyên cần, điểm số và tần

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
suất truy cập của sinh viên. Đồng thời, phải cung cấp giao diện tương tác thông minh cho giảng viên và sinh viên, hỗ trợ cảnh báo sớm thông qua thông báo tự động khi sinh viên chậm ngưỡng rủi ro.

Ngoài ra, hệ thống cần có khả năng phân tích dữ liệu chuyên sâu để cung cấp các báo cáo trực quan về tình trạng học tập, giúp giảng viên và cố vấn đưa ra các đánh giá công bằng, minh bạch. Việc tích hợp chatbot giúp việc tra cứu thông tin trở nên tức thì, giảm tải khối lượng công việc thủ công cho giảng viên và tăng cường sự kết nối giữa nhà trường và sinh viên.

Mục tiêu của dự án là xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ và cảnh báo sinh viên có nguy cơ thôi học dựa trên kiến trúc hiện đại, tích hợp Moodle API, sử dụng các thuật toán phân tích và AI để tối ưu hóa việc quản lý dữ liệu học tập.

Ý nghĩa của dự án không chỉ nằm ở khía cạnh kỹ thuật mà còn đóng góp lớn cho giáo dục. Hệ thống sẽ cải thiện chất lượng đào tạo, tăng tính minh bạch trong quản lý, hỗ trợ kịp thời cho sinh viên có nguy cơ bỏ học, và giảm tải khối lượng công việc hành chính cho giảng viên.

3.2. Phân tích yêu cầu người dùng

Hệ thống Chatbot cảnh báo học tập phục vụ bốn nhóm người dùng chính trong môi trường giáo dục đại học:

- **Giảng viên:** Theo dõi tiến độ, đánh giá kết quả học tập và phát hiện sớm các sinh viên có nguy cơ trong môn học mình phụ trách. Giảng viên cần công cụ tự động để tra cứu nhanh chóng và gửi cảnh báo kịp thời nhằm duy trì chất lượng giảng dạy.
- **Sinh viên:** Tra cứu điểm số, kiểm tra chuyên cần và nhận các thông báo nhắc nhở về tình trạng học tập cá nhân. Sinh viên cần một kênh tương tác trực tuyến tiện lợi để nắm bắt rủi ro và nhận gợi ý cải thiện kết quả học tập.
- **Cố vấn học tập:** Quản lý tổng quan tình hình học tập và mức độ tương tác của toàn bộ sinh viên trong lớp/khóa phụ trách. Cố vấn học tập cần hệ thống báo cáo

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

trực quan để theo dõi sát sao, kịp thời can thiệp và định hướng cho sinh viên có nguy cơ thôi học.

- **Quản trị viên (Admin):** Quản lý tổng thể hệ thống, thiết lập cấu hình các ngưỡng cảnh báo và giám sát quá trình đồng bộ dữ liệu. Quản trị viên cần các công cụ thống kê toàn cục để đảm bảo hệ thống vận hành trơn tru và hỗ trợ việc ra quyết định chiến lược của nhà trường.

3.2.1 Xác định tác nhân (Actors)

Bảng 3.1 Bảng Actors

Tác nhân	Mã vai trò	Mô tả
Sinh viên	STUDENT	Tra cứu điểm, cảnh báo, gợi ý cải thiện của bản thân
Giảng viên	LECTURER	Theo dõi lớp, lọc SV nguy cơ, nhắc nộp bài
Cố vấn học tập	ADVISER	Theo dõi lớp chủ nhiệm (cohort), SV không online
Quản trị viên	ADMIN	Cấu hình, đồng bộ, kiểm tra API, thống kê hệ thống

3.2.2 Yêu cầu của giảng viên

a) Tra cứu và quản lý thông tin học tập

- Tra cứu điểm số của sinh viên thông qua mã số sinh viên hoặc họ tên.
- Kiểm tra tình trạng nộp bài của sinh viên trong lớp (ví dụ: kiểm tra danh sách chưa nộp Assignment).

b) Theo dõi và xử lý cảnh báo rủi ro

- Lọc danh sách sinh viên có nguy cơ thôi học theo từng môn học (chỉ hiển thị các sinh viên thực sự vi phạm).
- Truy xuất danh sách sinh viên bị cảnh báo trên toàn hệ thống hoặc theo từng mức độ (Đỏ/Vàng).
- Thực hiện gửi thông báo cảnh báo thủ công trực tiếp đến sinh viên.
- Xác nhận và đánh dấu các cảnh báo đã được giảng viên xử lý.

c) Thống kê và hiển thị trực quan

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Xem thống kê tổng quan về số lượng sinh viên theo các mức độ cảnh báo (GREEN, YELLOW, RED).
- Nhận phản hồi từ hệ thống dưới dạng có cấu trúc rõ ràng (kèm icon trạng thái, bảng điểm, số liệu thống kê).
- Nhận thông báo tự động từ hệ thống khi phát hiện có sinh viên rơi vào mức cảnh báo đỏ trong môn học.

Bảng 3.2 Bảng yêu cầu của giảng viên

Tên	Mô tả	Intent tương ứng
Tra cứu điểm SV	Hỏi theo MSSV hoặc tên	QUERY_STUDENT_INFO_NLP
Lọc SV nguy cơ theo môn	VD: "Môn Java có bao nhiêu SV mức Đỏ?"	FILTER_COURSE_RISK
Kiểm tra bài chưa nộp	VD: "Lớp Java có ai chưa nộp Assignment?"	CHECK_SUBMISSIONS_AND_REMIND
Danh sách SV nguy cơ	Toàn hệ thống hoặc theo mức	QUERY_AT_RISK_LIST, QUERY_RED_ALERT_LIST
Gửi cảnh báo thủ công	Gửi notification đến SV	ACTION_SEND_WARNING_NOTIFICATION
Thống kê tổng quan	Số lượng GREEN/YELLOW/RED	QUERY_CLASS_SUMMARY
Xác nhận cảnh báo	Đánh dấu warning đã xử lý	API PUT /api/warnings/{id}/acknowledge

3.2.3 Yêu cầu của sinh viên

a) Tra cứu kết quả học tập cá nhân

- Xem bảng điểm chi tiết của các môn học đang tham gia.
- Kiểm tra thông tin điểm danh, chuyên cần và mức độ tham gia lớp học.
- Truy xuất báo cáo đầy đủ về toàn bộ tình trạng học tập hiện tại của bản thân.

b) Theo dõi cảnh báo và nhận tư vấn

- Kiểm tra trạng thái rủi ro cá nhân (xem bản thân có đang bị cảnh báo hay không).

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Nhận các gợi ý, định hướng từ chatbot để cải thiện điểm số và lộ trình học tập.

c) Ràng buộc bảo mật và quyền hạn

- Chỉ được truy xuất thông tin của chính mình; hệ thống tự động từ chối truy cập (Permission Denied) nếu tra cứu mã sinh viên khác.
- Tuyệt đối không được cấp quyền gọi các chức năng dành riêng cho giảng viên, cố vấn hoặc quản trị viên.

Bảng 3.3 Bảng yêu cầu của sinh viên

Tên	Ví dụ câu hỏi	Intent
Xem điểm	"Xem điểm của em", "Bảng điểm môn Java"	LIST_OWN_GRADES
Kiểm tra cảnh báo	"Em có bị cảnh báo không?"	CHECK_OWN_RISK_STATUS
Xem chuyên cần	"Chuyên cần của em"	QUERY_STUDENT_ATTENDANCE
Gợi ý cải thiện	"Làm sao để cải thiện điểm môn Java?"	GET_IMPROVEMENT_SUGGESTIONS
Báo cáo đầy đủ	"Tình trạng học tập của em"	QUERY_STUDENT_FULL_REPORT

3.2.4 Yêu cầu của cố vấn

a) Yêu cầu của Cố vấn học tập (Adviser)

- Xem báo cáo tổng quan về tình trạng cảnh báo rủi ro của toàn bộ sinh viên theo từng lớp phụ trách.
- Rà soát, tìm kiếm danh sách các sinh viên ngừng tương tác (ví dụ: không truy cập hệ thống trên 14 ngày) thông qua dữ liệu nhóm lớp.

Bảng 3.4 Bảng yêu cầu của cố vấn

Tên	Ví dụ	Intent
Tổng quan lớp	"Lớp DA22TTB có bao nhiêu SV bị cảnh báo?"	VIEW_CLASS_RISK_SUMMARY
SV ngừng tương tác	"SV không online trên 2 tuần"	FIND_INACTIVE_STUDENTS

3.2.5 Yêu cầu của quản trị viên (Admin)

b) Yêu cầu của Quản trị viên (Admin)

- Tùy chỉnh và cấu hình các ngưỡng cảnh báo rủi ro cho toàn hệ thống.
- Kích hoạt, quản lý và theo dõi quá trình đồng bộ dữ liệu từ hệ thống Moodle.
- Xem thống kê tổng quan về toàn bộ hoạt động của hệ thống chatbot.
- Kiểm tra trạng thái hoạt động của các API để đảm bảo tính ổn định.

c) Yêu cầu phi chức năng tổng hợp

- Đảm bảo hiệu năng hệ thống, thời gian phản hồi API chatbot dưới 5 giây; ưu tiên tối ưu truy vấn SQL.
- Giám sát tình trạng khả dụng của hệ thống qua Actuator và lưu trữ log file đầy đủ.
- Đảm bảo bảo mật bằng JWT, phân quyền chi tiết theo vai trò (Role-based) và cấu hình CORS độc lập.
- Thiết kế hệ thống theo kiến trúc Layered, Event-driven giúp dễ dàng mở rộng và bảo trì.
- Hỗ trợ triển khai linh hoạt trên các môi trường khác nhau (XAMPP native hoặc Docker).
- Tự động hóa quá trình đồng bộ dữ liệu định kỳ (mỗi 6 giờ) và thiết lập thời gian lưu trữ thông báo (90 ngày).
- Cung cấp tài liệu API rõ ràng (Swagger/OpenAPI) phục vụ tích hợp và kiểm thử.

Bảng 3.5 Bảng yêu cầu của admin

Tên	Intent	Mã UC
Cấu hình ngưỡng cảnh báo	CONFIG_WARNING_THRESHOLD	UC-AD-01
Đồng bộ dữ liệu Moodle	TRIGGER_MOODLE_SYNC	UC-AD-02
Kiểm tra API	ADMIN_CHECK_API_STATUS	UC-AD-03
Thống kê hệ thống	ADMIN_VIEW_SYSTEM_STATS	UC-AD-04

3.3. Lựa chọn công nghệ

Công nghệ được lựa chọn dựa trên yêu cầu chức năng, phi chức năng, tính phù hợp với môi trường giáo dục, khả năng tích hợp, hiệu suất, độ ổn định, cộng đồng hỗ trợ, khả năng mở rộng, dễ học, và chi phí.

3.3.1 Phía người dùng

- **HTML5/CSS3/JavaScript (thuần):** Nhẹ, dễ dàng nhúng trực tiếp dạng widget vào hệ thống Moodle hiện tại mà không cần thiết lập môi trường build phức tạp.
- **Chart.js:** Cung cấp giải pháp vẽ biểu đồ (doughnut, line) trực quan trên dashboard báo cáo, hiệu suất cao và tải qua CDN nhanh chóng.
- **Fetch API:** Chuẩn ES6+ hỗ trợ gọi REST API bất đồng bộ một cách đơn giản, tương tác mượt mà với backend.
- **SockJS/STOMP (WebSocket):** Hỗ trợ kết nối và đẩy thông báo theo thời gian thực (real-time) từ server xuống client.

3.3.2 Phía máy chủ

- **Java 17:** Phiên bản LTS ổn định, hiệu năng cao, hỗ trợ các tính năng mới (pattern matching), đặc biệt phù hợp cho phát triển hệ thống backend lớn.
- **Spring Boot 3.2.5:** Cung cấp hệ sinh thái toàn diện, cấu hình tối giản, dễ dàng phát triển RESTful API (Spring Web), bảo mật (Spring Security), WebSocket và giám sát (Actuator).
- **MySQL:** Đảm bảo tính toàn vẹn dữ liệu, lưu trữ quan hệ chặt chẽ. Việc dùng chung hệ quản trị với Moodle giúp dễ dàng truy vấn, trích xuất dữ liệu.
- **Spring Data JPA (Hibernate):** ORM mạnh mẽ giúp ánh xạ thực thể (Entity) với CSDL, quản lý schema tự động (ddl-auto) và rút gọn các thao tác truy vấn SQL.
- **Công cụ hỗ trợ (Lombok, dotenv-java, JWT):** Giảm thiểu code rườm rà (boilerplate), quản lý biến môi trường bảo mật, và xử lý mã hóa/giải mã token xác thực.
- **SpringDoc OpenAPI:** Tự động khởi tạo tài liệu API trực quan (Swagger UI) giúp dễ dàng kiểm thử và tích hợp giữa các thành phần.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

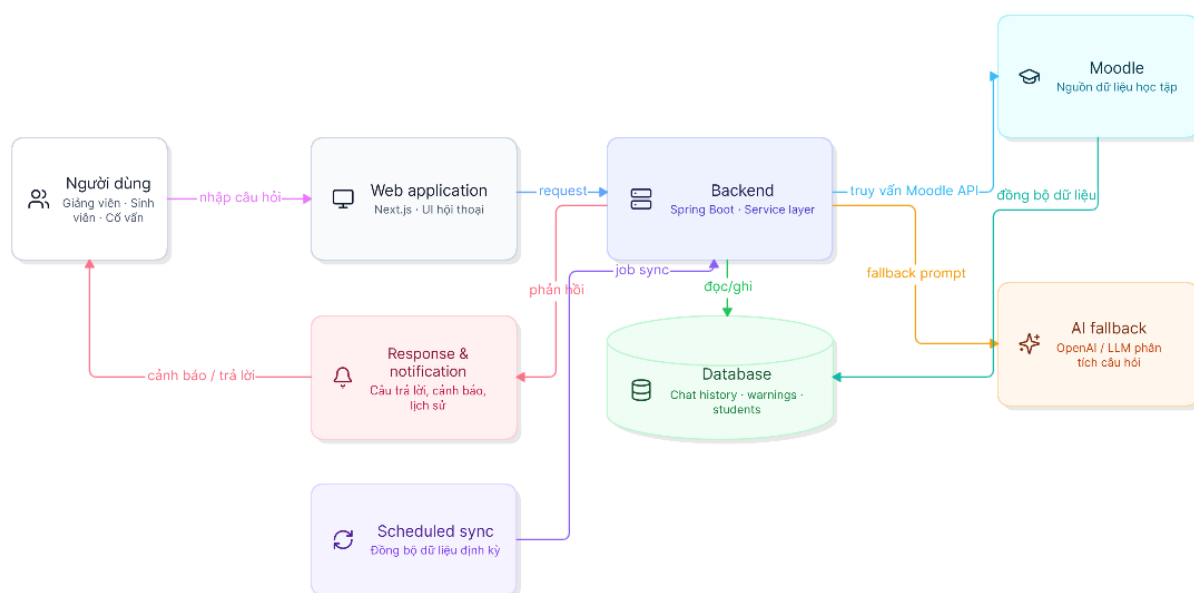
- **Maven:** Quản lý thư viện phụ thuộc chặt chẽ, tự động hóa quá trình đóng gói ứng dụng.

3.3.3 Tích hợp & dịch vụ ngoài

- **Moodle Web Services & Moodle DB:** Tích hợp trực tiếp để lấy dữ liệu cốt lõi (khóa học, điểm số, tỷ lệ hoàn thành), được trang bị cơ chế tự động thử lại (retry) khi gặp lỗi mạng.
- **OpenAI API / LLM:** Sử dụng model gpt-4o-mini thông qua Apache HttpClient để xử lý ngôn ngữ tự nhiên, nhận diện ý định và tạo phản hồi cho chatbot.
- **Docker:** Môi trường nhất quán, dễ triển khai, hỗ trợ học về containerization và microservices.

3.4. Thiết kế kiến trúc hệ thống tổng thể

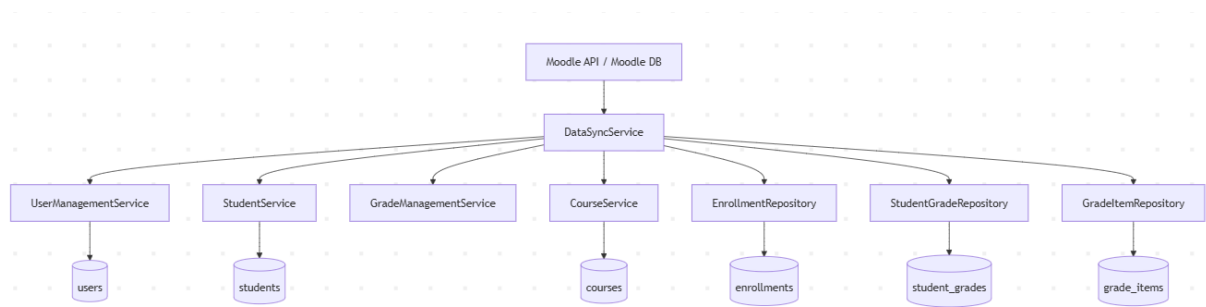
Hệ thống Chatbot cảnh báo học tập được thiết kế theo kiến trúc phân lớp (layered architecture) kết hợp với mô hình client-server, sử dụng các công nghệ hiện đại để đảm bảo tính mở rộng, hiệu suất cao và khả năng bảo trì tốt. Kiến trúc tổng thể được chia thành các lớp chức năng rõ ràng, mỗi lớp có trách nhiệm và nhiệm vụ cụ thể, tạo ra một hệ thống có cấu trúc logic và dễ dàng phát triển, triển khai và bảo trì. Hệ thống được thiết kế để hỗ trợ đa dạng đối tượng người dùng đồng thời, xử lý đồng bộ khối lượng dữ liệu học tập liên tục từ hệ thống Moodle cung cấp trải nghiệm tương tác mượt mà, phản hồi nhanh chóng theo thời gian thực.



Hình 3.1 Kiến trúc toàn bộ hệ thống

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Luồng đồng bộ dữ liệu từ moodle



Hình 3.2 Luồng đồng bộ dữ liệu từ moodle

Bước 1: Kết nối đến Moodle

Hệ thống khởi tạo kết nối đến Moodle thông qua cấu hình API hoặc cơ sở dữ liệu Moodle. Các thông tin kết nối như đường dẫn, token truy cập và thời gian chờ được khai báo trong file cấu hình của dự án.

Bước 2: Lấy danh sách người dùng và khóa học

Sau khi kết nối thành công, hệ thống tiến hành truy xuất dữ liệu người dùng, sinh viên, giảng viên và danh sách khóa học từ Moodle. Dữ liệu này được chuẩn hóa trước khi lưu vào cơ sở dữ liệu nội bộ.

Bước 3: Đồng bộ thông tin khóa học và thành phần điểm

Hệ thống tiếp tục lấy các thông tin liên quan đến khóa học, bao gồm tên môn học, mã môn, giảng viên phụ trách, các hạng mục điểm và trọng số của từng hạng mục. Dữ liệu này được lưu vào các bảng `courses` và `grade_items`.

Bước 4: Đồng bộ điểm số của sinh viên

Hệ thống thu thập điểm của từng sinh viên theo từng hạng mục điểm, sau đó ghi nhận vào bảng `student_grades`. Đồng thời, hệ thống xác định trạng thái nộp bài, điểm quy đổi và thời điểm chấm điểm nếu có.

Bước 5: Cập nhật dữ liệu đăng ký học

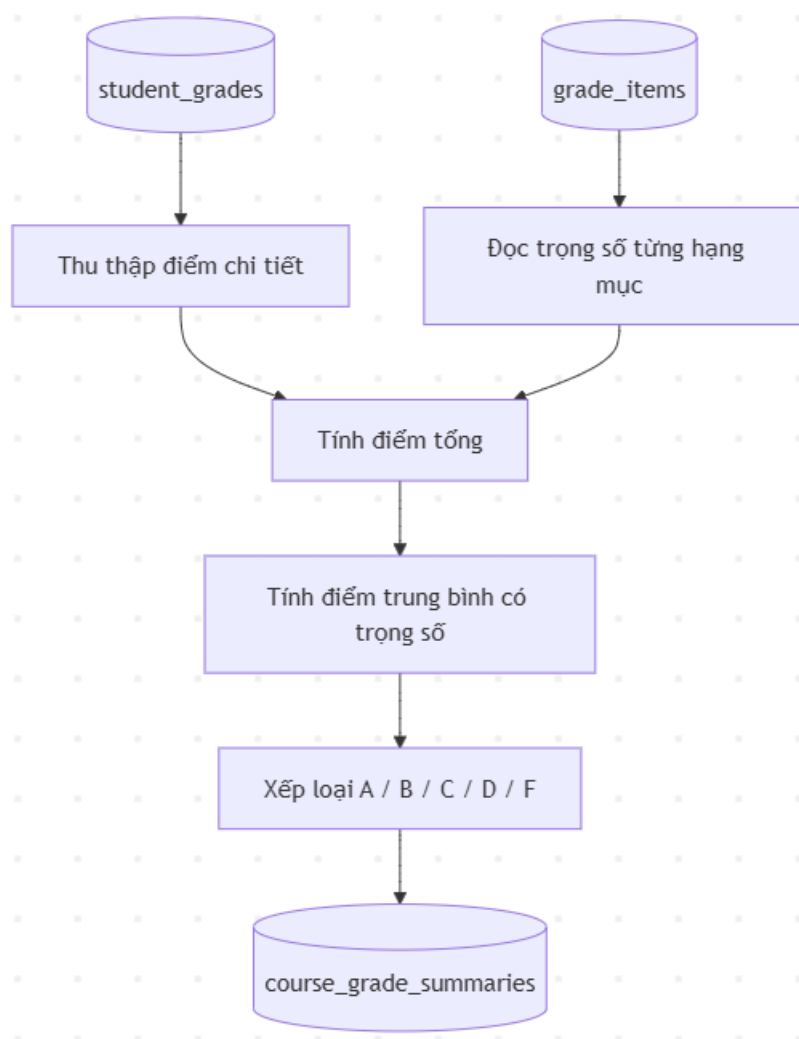
Thông tin sinh viên tham gia từng khóa học được đồng bộ vào bảng `enrollments`. Dữ liệu này bao gồm ngày đăng ký, lần truy cập cuối và tỷ lệ hoàn thành khóa học.

Bước 6: Cập nhật thời điểm đồng bộ

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Sau khi hoàn tất quá trình đồng bộ, hệ thống ghi nhận thời điểm đồng bộ gần nhất cho các thực thể liên quan nhằm phục vụ việc cập nhật dữ liệu ở các lần sau.

Luồng tính toán điểm tổng hợp của sinh viên



Hình 3.3 Luồng tính toán điểm tổng hợp của sinh viên

Bước 1: Thu thập dữ liệu điểm chi tiết

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Hệ thống lấy toàn bộ điểm số của sinh viên từ bảng student_grades và các thông tin cấu trúc điểm từ bảng grade_items.

Bước 2: Xác định trọng số từng hạng mục

Dựa trên loại hạng mục điểm và trọng số đã khai báo trong cơ sở dữ liệu, hệ thống xác định mức đóng góp của từng đầu điểm vào kết quả chung của khóa học.

Bước 3: Tính điểm tổng và điểm trung bình có trọng số

Hệ thống thực hiện tính toán tổng điểm, điểm trung bình có trọng số và các chỉ số liên quan như số lượng đầu điểm hoàn thành, nộp đúng hạn và nộp trễ.

Bước 4: Xếp loại học lực

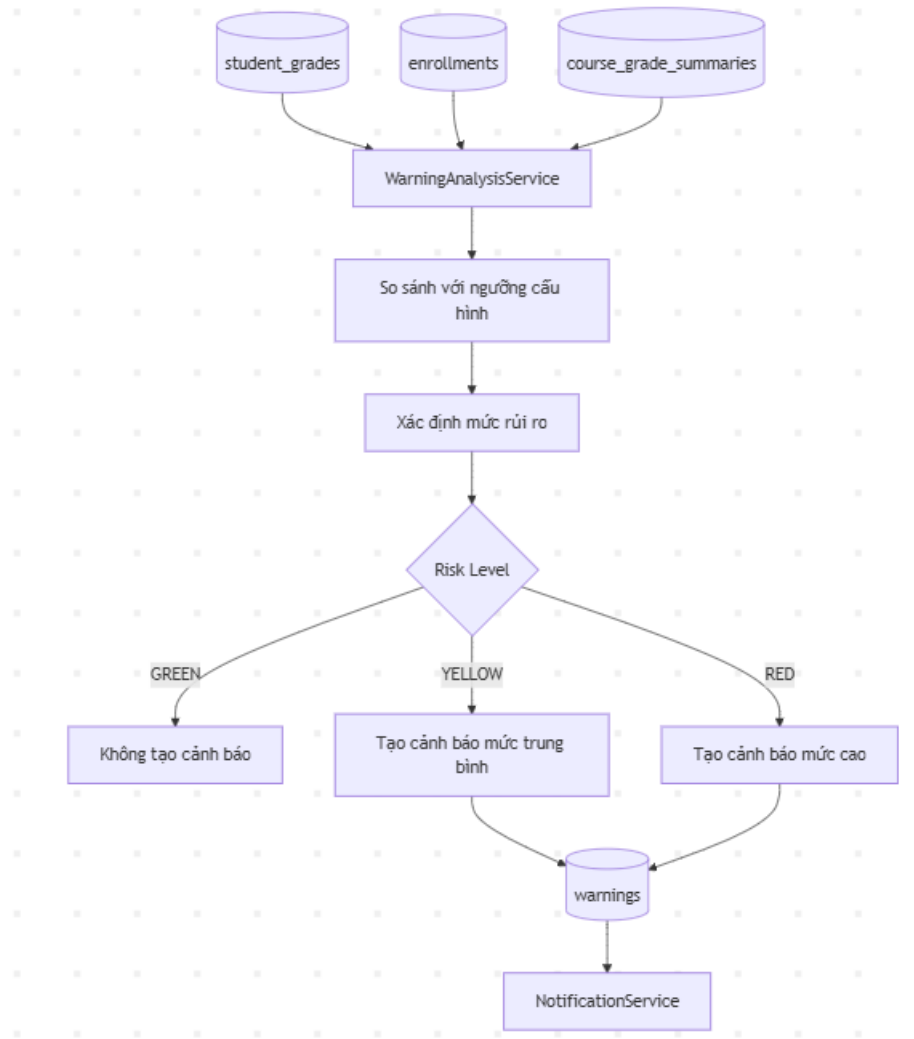
Từ điểm trung bình có trọng số, hệ thống phân loại kết quả học tập của sinh viên theo thang chữ cái như A, B, C, D hoặc F.

Bước 5: Lưu kết quả tổng hợp

Toàn bộ kết quả được lưu vào bảng course_grade_summaries để phục vụ cho việc tra cứu nhanh, thống kê và báo cáo sau này.

Luồng phát hiện và tạo cảnh báo rủi ro

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học



Hình 3.4 Luồng phát hiện và tạo cảnh báo rủi ro

Bước 1: Thu thập dữ liệu đầu vào

Hệ thống sử dụng dữ liệu từ các bảng student_grades, enrollments và course_grade_summaries để phân tích tình trạng học tập của sinh viên.

Bước 2: Đối chiếu với ngưỡng cảnh báo

Hệ thống so sánh các chỉ số như điểm trung bình, tỷ lệ chuyên cần, tỷ lệ hoàn thành và số ngày không truy cập với các ngưỡng được cấu hình trong ứng dụng.

Bước 3: Xác định mức độ rủi ro

Dựa trên kết quả so sánh, hệ thống phân loại sinh viên vào một trong ba mức độ rủi ro: GREEN, YELLOW hoặc RED.

Bước 4: Sinh cảnh báo

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Nếu sinh viên có dấu hiệu học tập không tốt, hệ thống tạo bản ghi cảnh báo trong bảng warnings. Bản ghi này lưu đầy đủ lý do cảnh báo, điểm trung bình, tỷ lệ chuyên cần, tỷ lệ hoàn thành và số ngày không truy cập.

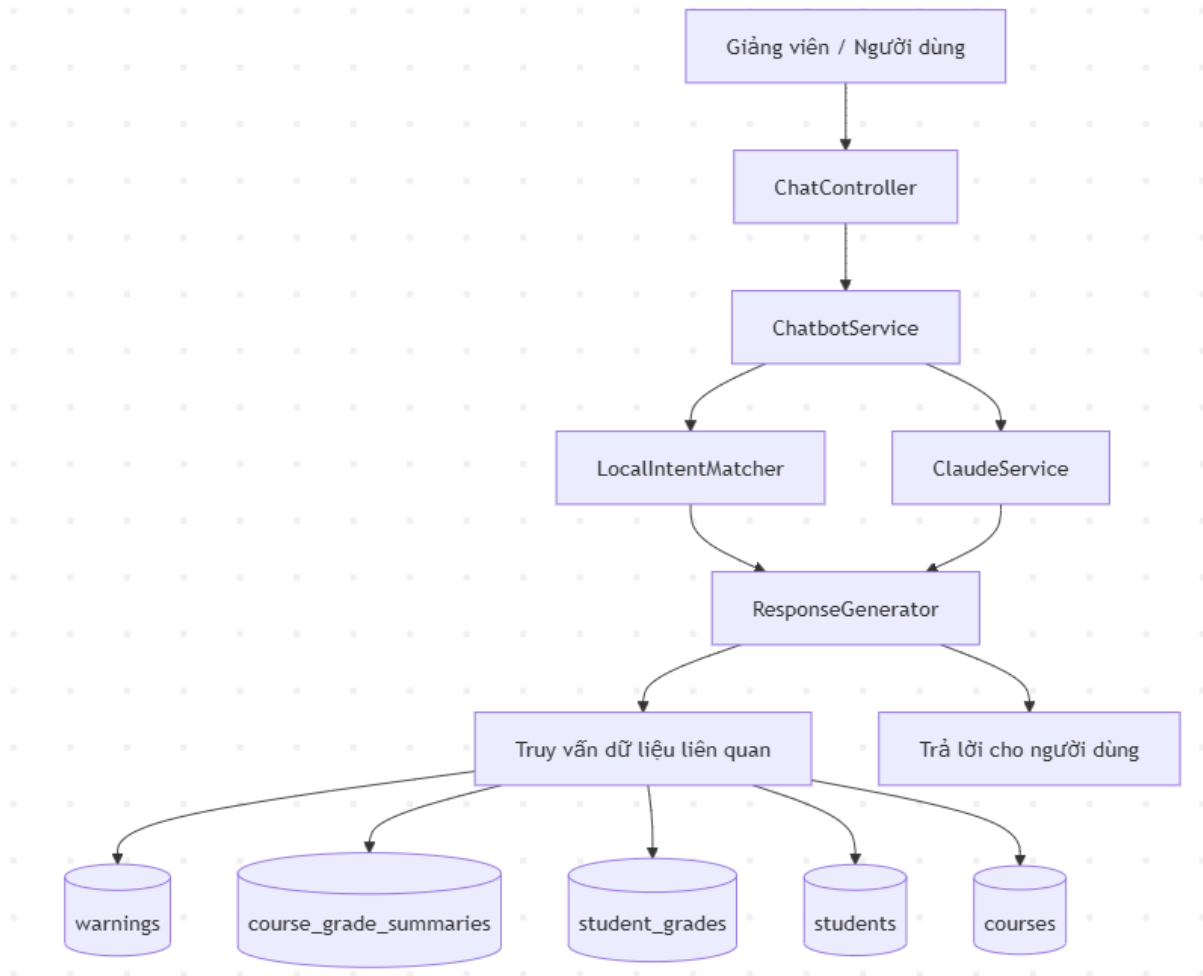
Bước 5: Gửi thông báo cho giảng viên

Sau khi cảnh báo được tạo, hệ thống có thể gửi thông báo qua các kênh hỗ trợ như WebSocket hoặc bộ phận thông báo nội bộ để giảng viên kịp thời nắm bắt tình hình.

Bước 6: Theo dõi trạng thái xử lý

Giảng viên có thể xác nhận, theo dõi hoặc xử lý cảnh báo. Trạng thái này được cập nhật lại trong cơ sở dữ liệu để phục vụ các lần truy vấn tiếp theo.

Luồng chatbot tư vấn và trả lời câu hỏi



Hình 3.5 Luồng chatbot tư vấn và trả lời câu hỏi

Bước 1: Người dùng gửi câu hỏi

Giảng viên nhập câu hỏi thông qua giao diện chatbot, ví dụ như yêu cầu xem danh sách sinh viên nguy cơ, thống kê học tập hoặc tình trạng của một sinh viên cụ thể.

Bước 2: Tiếp nhận yêu cầu

Hệ thống nhận nội dung câu hỏi thông qua ChatController và chuyển sang ChatbotService để xử lý.

Bước 3: Nhận diện ý định

Hệ thống sử dụng LocalIntentMatcher hoặc mô hình ngôn ngữ tích hợp để xác định ý định của câu hỏi, chẳng hạn như tra cứu trạng thái sinh viên, thống kê lớp học hoặc xem cảnh báo.

Bước 4: Truy vấn dữ liệu liên quan

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Dựa trên ý định đã nhận diện, hệ thống truy vấn các bảng dữ liệu phù hợp như warnings, course_grade_summaries, student_grades, students hoặc courses.

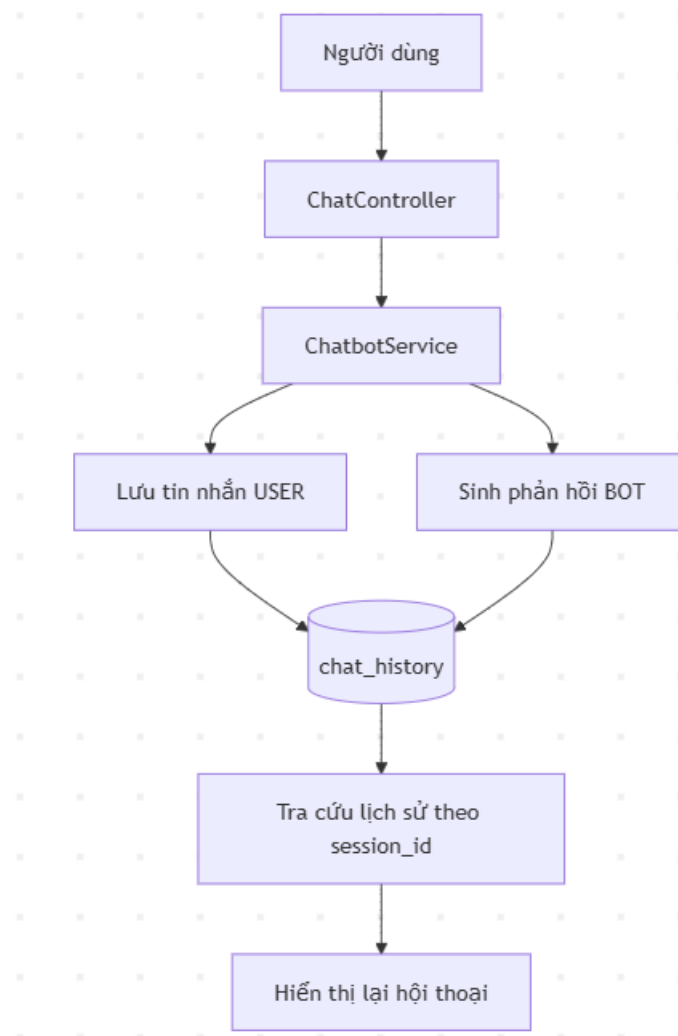
Bước 5: Sinh câu trả lời

Hệ thống tổng hợp dữ liệu truy vấn được và tạo phản hồi tự nhiên thông qua ResponseGenerator hoặc mô hình AI tích hợp.

Bước 6: Lưu lịch sử hội thoại

Toàn bộ nội dung câu hỏi và câu trả lời được lưu vào bảng chat_history để phục vụ truy vết, thống kê và cải thiện chất lượng chatbot.

Luồng quản lý lịch sử chat



Hình 3.6 Luồng quản lý lịch sử chat

Bước 1: Ghi nhận phiên làm việc

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
Mỗi lần người dùng bắt đầu trò chuyện, hệ thống tạo hoặc sử dụng một session_id để phân biệt từng phiên hội thoại.

Bước 2: Lưu tin nhắn của người dùng

Nội dung câu hỏi được lưu vào bảng chat_history cùng với vai trò USER, thời điểm tạo và ý định nếu đã nhận diện được.

Bước 3: Lưu phản hồi của chatbot

Sau khi chatbot tạo câu trả lời, hệ thống tiếp tục lưu bản ghi phản hồi với vai trò BOT.

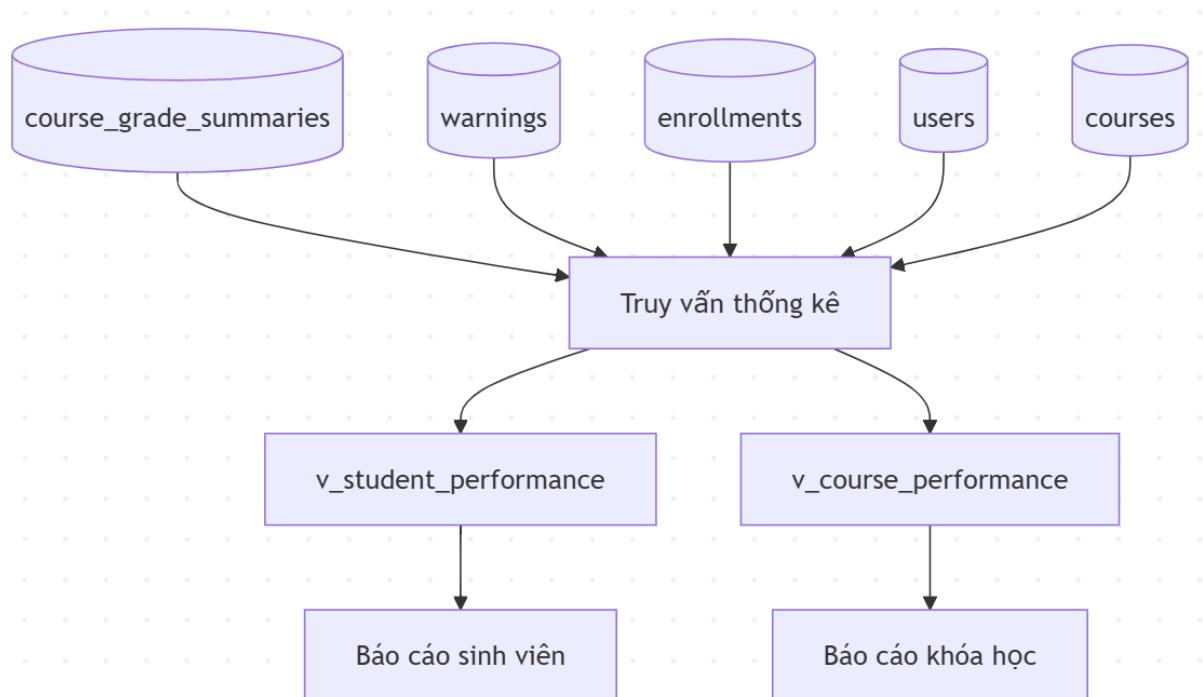
Bước 4: Tra cứu lại lịch sử

Khi cần kiểm tra nội dung trao đổi trước đó, hệ thống truy vấn bảng chat_history theo session_id hoặc lecturer_id.

Bước 5: Phục vụ cải tiến hệ thống

Dữ liệu lịch sử chat được sử dụng để phân tích mẫu câu hỏi phổ biến, đánh giá độ chính xác của chatbot và cải tiến khả năng phản hồi trong tương lai.

Luồng tra cứu và báo cáo



Hình 3.7 Luồng tra cứu và báo cáo

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bước 1: Truy vấn dữ liệu tổng hợp

Hệ thống lấy dữ liệu từ các bảng tổng hợp như `course_grade_summaries` và `warnings` để phục vụ báo cáo.

Bước 2: Tính toán chỉ số thống kê

Hệ thống xác định các chỉ số như số lượng sinh viên theo từng mức rủi ro, điểm trung bình lớp, tỷ lệ hoàn thành và số sinh viên cần cảnh báo.

Bước 3: Hiển thị báo cáo

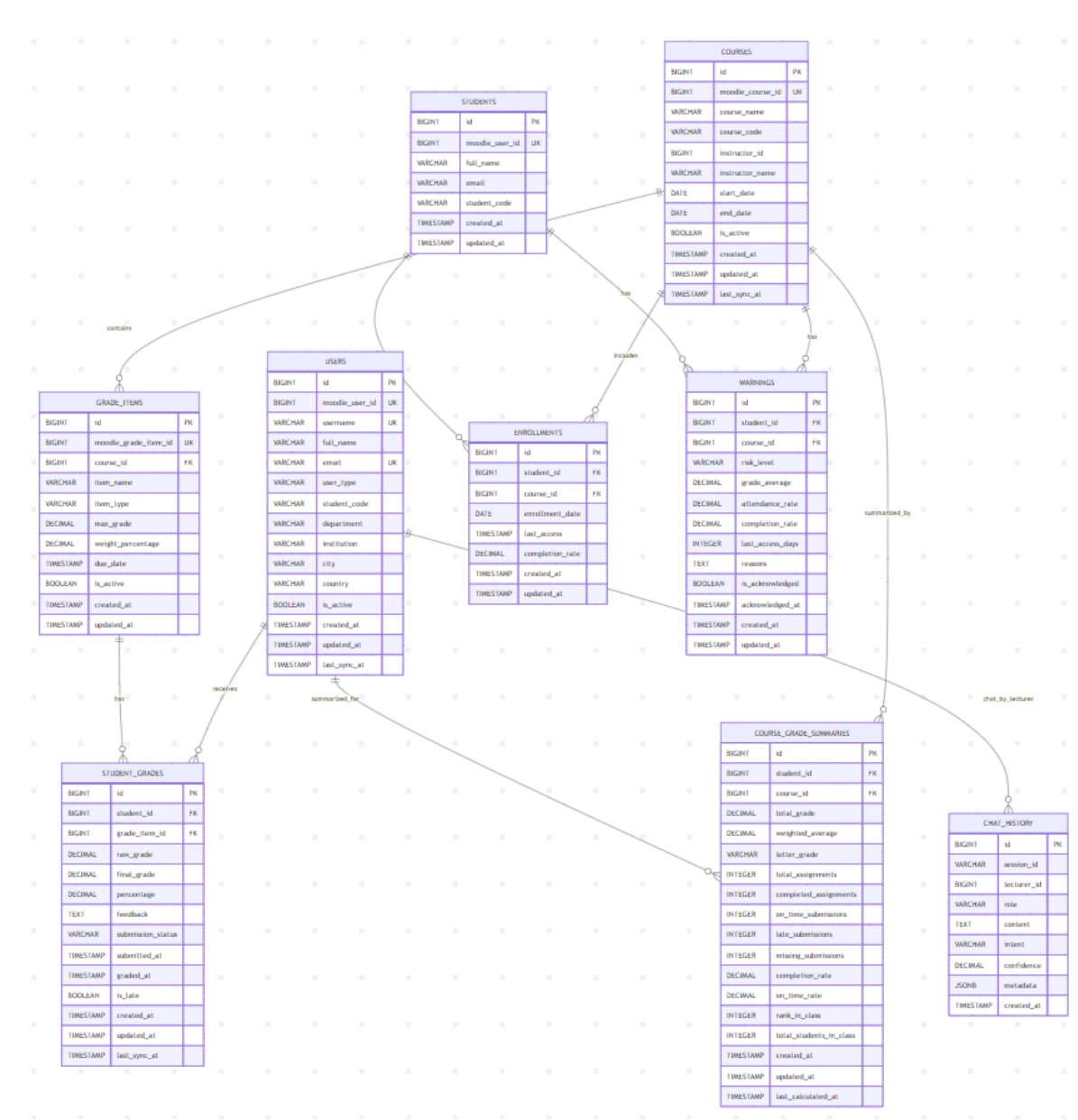
Kết quả được hiển thị trên giao diện dưới dạng bảng, số liệu tổng quan hoặc biểu đồ trực quan.

Bước 4: Hỗ trợ giảng viên ra quyết định

Thông qua báo cáo tổng hợp, giảng viên có thể nhận biết nhanh lớp học nào đang có nhiều sinh viên yếu, môn học nào có tỷ lệ hoàn thành thấp hoặc sinh viên nào cần được hỗ trợ sớm.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

3.5. Thiết kế cơ sở dữ liệu



Hình 3.8 Lược đồ cơ sở dữ liệu

- Mô tả bảng USER(Người dùng)

Bảng 3.6 Mô tả bảng User

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã người dùng nội bộ.
2	moodle_user_id	BIGINT	Unique, NOT NULL	ID người dùng từ Moodle.
3	username	VARCHAR(100)	Unique, NOT NULL	Tên đăng nhập.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

4	full_name	VARCHAR(200)	NOT NULL	Họ và tên đầy đủ.
5	email	VARCHAR(200)	Unique, NOT NULL	Địa chỉ email.
6	user_type	VARCHAR(20)	CHECK	Loại người dùng: STUDENT / TEACHER / ADMIN.
7	student_code	VARCHAR(50)	Nullable	Mã sinh viên, chỉ có ý nghĩa với STUDENT.
8	department	VARCHAR(200)	Nullable	Khoa / bộ môn.
9	institution	VARCHAR(200)	Nullable	Đơn vị / trường / cơ sở.
10	city	VARCHAR(100)	Nullable	Thành phố.
11	country	VARCHAR(10)	Nullable	Quốc gia.
12	is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động.
13	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
14	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.
15	last_sync_at	TIMESTAMP	Nullable	Lần đồng bộ Moodle gần nhất.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Mô tả bảng Student (Sinh viên):

Bảng 3.7 Mô tả bảng Student

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã sinh viên nội bộ.
2	moodle_user_id	BIGINT	Unique, NOT NULL	ID sinh viên từ Moodle.
3	full_name	VARCHAR(200)	NOT NULL	Họ và tên.
4	email	VARCHAR(200)	NOT NULL	Email.
5	student_code	VARCHAR(50)	Nullable	Mã sinh viên.
6	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
7	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.

- Mô tả bảng Course (Khóa học):

Bảng 3.8 Mô tả bảng Course

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã khóa học nội bộ.
2	moodle_course_id	BIGINT	Unique, NOT NULL	ID khóa học từ Moodle.
3	course_name	VARCHAR(500)	NOT NULL	Tên khóa học.
4	course_code	VARCHAR(50)	Nullable	Mã lớp/môn.
5	instructor_id	BIGINT	Nullable	ID giảng viên phụ trách.
6	instructor_name	VARCHAR(200)	Nullable	Tên giảng viên phụ trách.
7	start_date	DATE	Nullable	Ngày bắt đầu.
8	end_date	DATE	Nullable	Ngày kết thúc.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

9	is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động.
10	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
11	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.
12	last_sync_at	TIMESTAMP	Nullable	Lần đồng bộ gần nhất.

- Mô tả bảng grade_items (Lưu từng thành phần điểm của khóa học như bài tập, quiz, thi giữa kỳ, thi cuối kỳ.)

Bảng 3.9 Mô tả bảng grade_items

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã hạng mục điểm.
2	moodle_grade_item_id	BIGINT	Unique, NOT NULL	ID hạng mục điểm từ Moodle.
3	course_id	BIGINT	FK, NOT NULL	Khóa học chứa hạng mục điểm.
4	item_name	VARCHAR(500)	NOT NULL	Tên bài / đầu điểm.
5	item_type	VARCHAR(50)	CHECK	Loại hạng mục: ASSIGNMENT, QUIZ, MIDTERM_EXAM, FINAL_EXAM, PROJECT, PRESENTATION, LAB, ATTENDANCE, OTHER.
6	max_grade	DECIMAL(10,2)	NOT NULL	Điểm tối đa.
7	weight_percentage	DECIMAL(5,2)	Nullable	Tỷ trọng điểm.
8	due_date	TIMESTAMP	Nullable	Hạn nộp / hạn thi.
9	is_active	BOOLEAN	Default TRUE	Trạng thái sử dụng.
10	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

11	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.
----	------------	-----------	------------------------------	---------------------

- Mô tả bảng Studen_grades (Lưu điểm chi tiết của từng sinh viên cho từng hạng mục điểm)

Bảng 3.10 Mô tả bảng Studen_grades

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã bản ghi điểm.
2	student_id	BIGINT	FK, NOT NULL	Sinh viên được chấm điểm.
3	grade_item_id	BIGINT	FK, NOT NULL	Hạng mục điểm tương ứng.
4	raw_grade	DECIMAL(10,2)	Nullable	Điểm thô từ Moodle.
5	final_grade	DECIMAL(10,2)	Nullable	Điểm cuối cùng sau quy đổi.
6	percentage	DECIMAL(5,2)	Nullable	Phần trăm hoàn thành / quy đổi.
7	feedback	TEXT	Nullable	Nhận xét của giảng viên.
8	submission_status	VARCHAR(50)	CHECK	Trạng thái nộp: NOT_SUBMITTED, SUBMITTED, GRADED, RESUBMITTED, LATE.
9	submitted_at	TIMESTAMP	Nullable	Thời điểm nộp.
10	graded_at	TIMESTAMP	Nullable	Thời điểm chấm.
11	is_late	BOOLEAN	Default FALSE	Có nộp trễ hay không.
12	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
13	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.
14	last_sync_at	TIMESTAMP	Nullable	Lần đồng bộ gần nhất.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Mô tả bảng Course_grade_sumaries (Tổng hợp kết quả của một sinh viên trong một khóa học, gồm điểm trung bình, tỷ lệ hoàn thành và xếp hạng.)

Bảng 3.11 Mô tả bảng Course_grade_sumaries

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã bản ghi tổng hợp.
2	student_id	BIGINT	FK, NOT NULL	Sinh viên được tổng hợp.
3	course_id	BIGINT	FK, NOT NULL	Khóa học tương ứng.
4	total_grade	DECIMAL(10,2)	Nullable	Tổng điểm.
5	weighted_average	DECIMAL(10,2)	Nullable	Điểm trung bình có trọng số.
6	letter_grade	VARCHAR(5)	Nullable	Xếp loại A/B/C/D/F.
7	total_assignments	INTEGER	Default 0	Tổng số đầu điểm.
8	completed_assignments	INTEGER	Default 0	Số đầu điểm đã hoàn thành.
9	on_time_submissions	INTEGER	Default 0	Số lần nộp đúng hạn.
10	late_submissions	INTEGER	Default 0	Số lần nộp trễ.
11	missing_submissions	INTEGER	Default 0	Số lần chưa nộp.
12	completion_rate	DECIMAL(5,2)	Nullable	Tỷ lệ hoàn thành.
13	on_time_rate	DECIMAL(5,2)	Nullable	Tỷ lệ nộp đúng hạn.
14	rank_in_class	INTEGER	Nullable	Xếp hạng trong lớp.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

15	total_students_in_class	INTEGER	Nullable	Tổng số sinh viên trong lớp.
16	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
17	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.
18	last_calculated_at	TIMESTAMP	Nullable	Thời điểm tính lại gần nhất.

- Mô tả bảng enrollments (Lưu quan hệ đăng ký khóa học của sinh viên, kèm trạng thái tiến độ và thời điểm truy cập gần nhất.)

Bảng 3.12 Mô tả bảng enrollments

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã đăng ký.
2	student_id	BIGINT	FK, NOT NULL	Sinh viên đăng ký.
3	course_id	BIGINT	FK, NOT NULL	Khóa học được đăng ký.
4	enrollment_date	DATE	NOT NULL	Ngày đăng ký.
5	last_access	TIMESTAMP	Nullable	Lần truy cập cuối.
6	completion_rate	DECIMAL(5,2)	Default 0.00	Tỷ lệ hoàn thành khóa học.
7	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
8	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Mô tả bảng Warnings (Lưu các cảnh báo học tập theo sinh viên và khóa học, phục vụ phát hiện sớm sinh viên có nguy cơ.)

Bảng 3.13 Mô tả bảng Warnings

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã cảnh báo.
2	student_id	BIGINT	FK, NOT NULL	Sinh viên bị cảnh báo.
3	course_id	BIGINT	FK, NOT NULL	Khóa học liên quan.
4	risk_level	VARCHAR(10)	CHECK	Mức độ rủi ro: GREEN / YELLOW / RED.
5	grade_average	DECIMAL(5,2)	Nullable	Điểm trung bình.
6	attendance_rate	DECIMAL(5,2)	Nullable	Tỷ lệ chuyên cần.
7	completion_rate	DECIMAL(5,2)	Nullable	Tỷ lệ hoàn thành.
8	last_access_days	INTEGER	Nullable	Số ngày kể từ lần truy cập cuối.
9	reasons	TEXT	Nullable	Danh sách lý do cảnh báo.
10	is_acknowledged	BOOLEAN	Default FALSE	Đã được giảng viên xác nhận hay chưa.
11	acknowledged_at	TIMESTAMP	Nullable	Thời điểm xác nhận.
12	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.
13	updated_at	TIMESTAMP	Default current_timestamp	Thời điểm cập nhật.

- Mô tả bảng Chat_history (Lưu toàn bộ lịch sử hội thoại giữa giảng viên và chatbot)

Bảng 3.14 Mô tả bảng Review ai

STT	Thuộc tính	Kiểu dữ liệu	Ràng buộc	Diễn giải
1	id	BIGSERIAL	Primary key	Mã bản ghi chat.
2	session_id	VARCHAR(100)	NOT NULL	Mã phiên chat.
3	lecturer_id	BIGINT	Nullable	ID giảng viên (nếu có).
4	role	VARCHAR(20)	CHECK	Vai trò tin nhắn: USER / BOT.
5	content	TEXT	NOT NULL	Nội dung tin nhắn.
6	intent	VARCHAR(100)	Nullable	Ý định nhận diện được.
7	confidence	DECIMAL(5,4)	Nullable	Độ tin cậy của intent.
8	metadata	JSONB	Nullable	Dữ liệu bổ sung (nếu có).
9	created_at	TIMESTAMP	Default current_timestamp	Thời điểm tạo.

3.6. Triển khai Chatbot

Trong hệ thống cảnh báo sớm kết quả học tập, chatbot được xây dựng như một lớp giao tiếp thông minh giữa người dùng và cơ sở dữ liệu của hệ thống. Thay vì yêu cầu người dùng thao tác qua nhiều màn hình báo cáo phức tạp, chatbot cho phép giảng viên và người học truy vấn nhanh các thông tin như tình trạng học tập, mức độ rủi ro, điểm số, tiến độ nộp bài và các thông kê liên quan.

Hệ thống chatbot trong đề tài được triển khai theo hướng kết hợp giữa nhận diện ý định cục bộ và sinh phản hồi bằng mô hình ngôn ngữ Claude. Cách tiếp cận này giúp hệ thống vừa đảm bảo tốc độ xử lý nhanh đối với các truy vấn phổ biến, vừa có khả năng tạo ra câu trả lời tự nhiên, linh hoạt và dễ hiểu.

3.6.1 Mục tiêu triển khai chatbot

Chatbot trong hệ thống được xây dựng với các mục tiêu chính sau:

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Hỗ trợ giảng viên và người dùng truy vấn nhanh thông tin học tập.
- Tự động nhận diện ý định câu hỏi của người dùng.
- Trả lời bằng ngôn ngữ tự nhiên, thân thiện và dễ hiểu.
- Tích hợp được với dữ liệu nội bộ như sinh viên, khóa học, điểm số và cảnh báo.
- Lưu lại toàn bộ lịch sử trao đổi để phục vụ tra cứu và cải tiến hệ thống.

Đặc biệt, chatbot không chỉ đơn thuần đóng vai trò trả lời câu hỏi, mà còn là một công cụ hỗ trợ ra quyết định cho giảng viên trong việc phát hiện sớm sinh viên có nguy cơ học tập thấp.

3.6.2 Kiến trúc triển khai Chatbot

Kiến trúc chatbot của hệ thống được xây dựng theo mô hình nhiều tầng, gồm các thành phần chính:

- **Giao diện người dùng:** nơi giảng viên hoặc sinh viên nhập câu hỏi.
- **ChatController:** tiếp nhận yêu cầu và chuyển cho tầng xử lý nghiệp vụ.
- **ChatbotService:** điều phối toàn bộ quá trình xử lý hội thoại.
- **LocalIntentMatcher:** nhận diện ý định từ câu hỏi người dùng bằng các mẫu từ khóa và biểu thức chính quy.
- **ClaudeService:** kết nối tới API Claude để sinh phản hồi tự nhiên.
- **ResponseGenerator:** tổng hợp dữ liệu nghiệp vụ và xây dựng nội dung trả lời.
- **ChatHistoryRepository / chat_history:** lưu lịch sử hội thoại.

Mô hình này giúp hệ thống có thể xử lý theo hai hướng:

1. Nếu câu hỏi thuộc nhóm đơn giản, lặp lại nhiều lần như chào hỏi, hỏi trợ giúp, xem điểm, xem cảnh báo, hệ thống có thể xử lý nhanh bằng bộ nhận diện ý định cục bộ.
2. Nếu câu hỏi cần diễn giải tự nhiên hơn hoặc cần phản hồi linh hoạt, hệ thống có thể gọi Claude để sinh câu trả lời phù hợp.

3.6.2.1 Cơ chế nhận định ý định

Một điểm nổi bật trong phần triển khai chatbot của đề tài là sử dụng lớp nhận diện ý định cục bộ để phân loại nhanh yêu cầu của người dùng. Trong repo, phần này được xây dựng bằng các biểu thức chính quy để nhận biết các nhóm câu hỏi phổ biến như:

- chào hỏi, tạm biệt, cảm ơn;
- xem điểm, xem bảng điểm;
- tra cứu tình trạng học tập;
- xem sinh viên có nguy cơ;
- truy vấn theo MSSV hoặc theo lớp;
- yêu cầu đồng bộ dữ liệu;
- kiểm tra trạng thái API hoặc trạng thái hệ thống.

Cách làm này có ưu điểm là đơn giản, nhanh và phù hợp với các truy vấn mang tính nghiệp vụ rõ ràng. Khi câu hỏi của người dùng khớp với một mẫu đã biết, hệ thống sẽ gán cho nó một intent tương ứng, đồng thời trích xuất thêm các thực thể như:

- mã số sinh viên;
- mã lớp;
- mức độ rủi ro;
- số ngày không hoạt động;
- tên môn học.

Nhờ đó, chatbot không chỉ đọc câu hỏi mà còn hiểu được đối tượng và ngữ cảnh cần truy vấn.

3.6.2.2 Tích hợp AI vào hệ thống

Claude được sử dụng để tăng khả năng sinh ngôn ngữ tự nhiên cho chatbot. Thay vì trả về kết quả khô khan dưới dạng dữ liệu máy đọc, hệ thống có thể diễn đạt câu trả lời thành ngôn ngữ gần với cách con người giao tiếp.

Trong file cấu hình application.yml, hệ thống đã khai báo các tham số cho Claude như:

- api.key: khóa API lấy từ biến môi trường;

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- url: đường dẫn tới endpoint chat completions;
- model: tên mô hình sử dụng;
- timeout: thời gian chờ phản hồi.

Việc đưa khóa API ra biến môi trường giúp tránh lộ thông tin nhạy cảm trong mã nguồn. Ngoài ra, việc cấu hình model và timeout cũng giúp hệ thống dễ dàng thay đổi hoặc tối ưu trong quá trình triển khai.

Trong kiến trúc tổng thể, Claude thường được dùng trong các trường hợp:

- cần câu trả lời tự nhiên, mạch lạc;
- cần giải thích kết quả phân tích;
- cần diễn đạt lại dữ liệu truy vấn từ hệ thống;
- cần hỗ trợ người dùng với ngôn ngữ linh hoạt hơn so với câu trả lời mẫu.

3.6.2.3 Luồng xử lý một yêu cầu Chatbot

Khi người dùng gửi một câu hỏi, hệ thống xử lý theo các bước sau:

Bước 1: Tiếp nhận yêu cầu

Người dùng nhập câu hỏi vào giao diện chatbot. Yêu cầu được gửi tới ChatController.

Bước 2: Phân tích câu hỏi

ChatbotService nhận nội dung câu hỏi và chuyển cho LocalIntentMatcher để xác định ý định ban đầu.

Bước 3: Xác định ngữ cảnh và quyền truy cập

Dựa vào vai trò của người dùng như sinh viên, giảng viên hay quản trị viên, hệ thống kiểm tra xem câu hỏi có thuộc phạm vi cho phép hay không. Điều này giúp hạn chế truy vấn ngoài quyền.

Bước 4: Truy vấn dữ liệu nội bộ

Nếu câu hỏi liên quan đến điểm số, cảnh báo hoặc khóa học, hệ thống truy xuất dữ liệu từ các bảng

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học
như student_grades, course_grade_summaries, warnings, students, courses hoặc chat_history.

Bước 5: Sinh phản hồi

Sau khi có dữ liệu cần thiết, hệ thống chuyển thông tin cho ResponseGenerator hoặc Claude để tạo câu trả lời.

Bước 6: Ghi lịch sử hội thoại

Toàn bộ câu hỏi và câu trả lời được lưu vào bảng chat_history để phục vụ tra cứu sau này.

Luồng xử lý này đảm bảo chatbot vừa có khả năng trả lời nhanh, vừa có thể mở rộng để xử lý các truy vấn phức tạp hơn trong tương lai.

CHƯƠNG 4. CÀI ĐẶT HỆ THỐNG VÀ KẾT QUẢ THỰC NGHIỆM

4.1. Xây dựng các API chức năng chính

Bảng 4.1. Bảng mô tả API đồng bộ người dùng

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/users/sync
Content-Type	application/json
Controller	UserController.syncAllUsers()
Diễn giải	API cho phép đồng bộ toàn bộ người dùng từ Moodle về hệ thống.

Bảng 4.2. Bảng mô tả API lấy danh sách sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/students
Content-Type	application/json
Controller	UserController.getAllStudents()
Diễn giải	API trả về danh sách tất cả sinh viên trong hệ thống.

Bảng 4.3. Bảng mô tả API lấy danh sách giáo viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/teachers
Content-Type	application/json
Controller	UserController.getAllTeachers()
Diễn giải	API trả về danh sách tất cả giáo viên trong hệ thống.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.4. Bảng mô tả API tìm sinh viên theo từ khóa

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/students/search?keyword={keyword}
Content-Type	application/json
Controller	UserController.searchStudents()
Diễn giải	API cho phép tìm kiếm sinh viên theo từ khóa nhập vào.

Bảng 4.5 Bảng mô tả API lấy sinh viên theo mã sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/students/code/{studentCode}
Content-Type	application/json
Controller	UserController.getUserByStudentCode()
Diễn giải	API cho phép lấy thông tin sinh viên theo mã sinh viên.

Bảng 4.6. Bảng mô tả API lấy người dùng theo Moodle ID

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/moodle/{moodleUserId}
Content-Type	application/json
Controller	UserController.getUserByMoodleId()
Diễn giải	API cho phép tra cứu người dùng theo ID đồng bộ từ Moodle.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.7. Bảng mô tả API thống kê người dùng

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/users/statistics
Content-Type	application/json
Controller	UserController.getUserStatistics()
Diễn giải	API trả về thống kê số lượng người dùng theo từng nhóm.

Bảng 4.8. Bảng mô tả API lấy danh sách sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/students
Content-Type	application/json
Controller	StudentController.getAllStudents()
Diễn giải	API trả về danh sách sinh viên hoặc kết quả tìm kiếm theo tên.

Bảng 4.9. Bảng mô tả API lấy chi tiết sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/students/{id}
Content-Type	application/json
Controller	StudentController.getStudentById()
Diễn giải	API cho phép lấy thông tin chi tiết của một sinh viên theo ID.

Bảng 4.10. Bảng mô tả API phân tích rủi ro sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/students/{id}/risk
Content-Type	application/json

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Controller	StudentController.getStudentRisk()
Diễn giải	API trả về kết quả phân tích mức độ rủi ro của sinh viên.

Bảng 4.11. Bảng mô tả API đồng bộ grade items

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/grades/sync/items/{courseId}
Content-Type	application/json
Controller	GradeController.syncGradeItems()
Diễn giải	API đồng bộ danh sách các đầu điểm của một khóa học.

Bảng 4.12. Bảng mô tả API đồng bộ điểm sinh viên

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/grades/sync/student/{studentId}/course/{courseId}
Content-Type	application/json
Controller	GradeController.syncStudentGrades()
Diễn giải	API đồng bộ điểm của một sinh viên trong một khóa học.

Bảng 4.13. Bảng mô tả API lấy báo cáo điểm chi tiết

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/student/{studentId}/course/{courseId}
Content-Type	application/json
Controller	GradeController.getStudentCourseReport()
Diễn giải	API trả về báo cáo điểm chi tiết của sinh viên trong khóa học.

Bảng 4.14. Bảng mô tả API lấy toàn bộ điểm của sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/student/{studentId}

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Content-Type	application/json
Controller	GradeController.getStudentGrades()
Diễn giải	API trả về toàn bộ điểm của sinh viên.

Bảng 4.15. Bảng mô tả API lấy tổng hợp điểm theo môn

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/summary/student/{studentId}
Content-Type	application/json
Controller	GradeController.getStudentSummaries()
Diễn giải	API lấy danh sách tổng hợp điểm theo từng khóa học của sinh viên.
Thuộc tính	Giá trị

Bảng 4.16. Bảng mô tả API lấy các đầu điểm của khóa học

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/items/course/{courseId}
Content-Type	application/json
Controller	GradeController.getGradeItemsByCourse()
Diễn giải	API lấy danh sách các grade item của khóa học.

Bảng 4.17. Bảng mô tả API lấy điểm trung bình sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/average/student/{studentId}
Content-Type	application/json
Controller	GradeController.getAverageGrade()

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Diễn giải	API trả về điểm trung bình của sinh viên.
-----------	---

Bảng 4.18. Bảng mô tả API lọc sinh viên hoàn thành thấp

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/low-completion/course/{courseId}?threshold=70
Content-Type	application/json
Controller	GradeController.getLowCompletionStudents()
Diễn giải	API lấy danh sách sinh viên có completion rate thấp hơn ngưỡng quy định.

Bảng 4.19. Bảng mô tả API xếp hạng sinh viên trong lớp

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/grades/ranking/course/{courseId}
Content-Type	application/json
Controller	GradeController.getClassRanking()
Diễn giải	API lấy xếp hạng sinh viên trong khóa học.

Bảng 4.20. Bảng mô tả API lấy danh sách cảnh báo

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/warnings
Content-Type	application/json
Controller	WarningController.getAllWarnings()
Diễn giải	API lấy danh sách cảnh báo chưa được xác nhận.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.21. Bảng mô tả API lấy cảnh báo mức đỏ

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/warnings/red
Content-Type	application/json
Controller	WarningController.getRedWarnings()
Diễn giải	API lấy danh sách cảnh báo ở mức RED.

Bảng 4.22. Bảng mô tả API thống kê cảnh báo

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/warnings/dashboard
Content-Type	application/json
Controller	WarningController.getDashboard()
Diễn giải	API trả về thống kê cảnh báo theo mức độ rủi ro.

Bảng 4.23. Bảng mô tả API xác nhận cảnh báo

Thuộc tính	Giá trị
Phương thức	PUT
Endpoint	/api/warnings/{id}/acknowledge?lecturerId={lecturerId}
Content-Type	application/json
Controller	WarningController.acknowledgeWarning()
Diễn giải	API cho phép giảng viên xác nhận một cảnh báo cụ thể.

Bảng 4.24. Bảng mô tả API tạo cảnh báo thử nghiệm

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/warnings/test-red
Content-Type	application/json
Controller	WarningController.testRedEvent()
Thuộc tính	Giá trị

Bảng 4.25. Bảng mô tả API chatbot gửi tin nhắn

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/chat/message
Content-Type	application/json
Controller	ChatController.sendMessage()
Diễn giải	API cho phép gửi tin nhắn vào chatbot để xử lý và sinh phản hồi.

Bảng 4.26. Bảng mô tả API lấy lịch sử chat

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/chat/history/{username}
Content-Type	application/json
Controller	ChatController.getChatHistory()
Diễn giải	API lấy lịch sử trò chuyện theo tên người dùng.

Bảng 4.27. Bảng mô tả API đồng bộ toàn bộ dữ liệu thủ công

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/sync/manual
Content-Type	application/json
Controller	SyncController.manualSync()
Diễn giải	API đồng bộ toàn bộ dữ liệu từ Moodle theo yêu cầu thủ công.

Bảng 4.28. Bảng mô tả API kiểm tra kết nối Moodle

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/sync/test-connection
Content-Type	application/json
Controller	SyncController.testConnection()
Diễn giải	API kiểm tra trạng thái kết nối đến Moodle API.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.29. Bảng mô tả API gửi thông báo thủ công

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/notifications/send
Content-Type	application/json
Controller	NotificationController.sendNotification()
Diễn giải	API gửi một thông báo thủ công tới người học.

Bảng 4.30. Bảng mô tả API gửi thông báo hàng loạt

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/notifications/batch
Content-Type	application/json
Controller	NotificationController.batchScan()
Diễn giải	API quét hàng loạt và gửi thông báo tự động.

Bảng 4.31. Bảng mô tả API lấy thông báo của người dùng

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/notifications/user/{userId}?limit=20
Content-Type	application/json
Controller	NotificationController.getUserMessages()
Diễn giải	API lấy danh sách thông báo của một người dùng cụ thể.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.32. Bảng mô tả API lấy điểm sinh viên từ Moodle

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/grades?studentId={studentId}&courseName={courseName}
Content-Type	application/json
Controller	MoodleRestController.getStudentGrades()
Diễn giải	API lấy điểm của một sinh viên theo từng khóa học trên Moodle.

Bảng 4.33. Bảng mô tả API lấy chuyên cần sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/attendance?studentId={studentId}&courseName={courseName}
Content-Type	application/json
Controller	MoodleRestController.getAttendance()
Diễn giải	API lấy thông tin chuyên cần và lần truy cập cuối của sinh viên.

Bảng 4.34. Bảng mô tả API lấy trạng thái sinh viên

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/{studentId}/status
Content-Type	application/json

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Controller	MoodleRestController.getStudentStatus()
Diễn giải	API trả về trạng thái phân tích rủi ro của sinh viên.

Bảng 4.35. Bảng mô tả API lọc sinh viên nguy cơ

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/at-risk?riskLevel=&minAvgGrade=50&maxInactiveCourses=1&inactiveDays=14&onlyViolations=true
Content-Type	application/json
Controller	MoodleRestController.getAtRiskStudents()
Diễn giải	API lọc danh sách sinh viên có nguy cơ theo tiêu chí đặt trước.

Bảng 4.36. Bảng mô tả API tổng quan lớp học

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/class-overview
Content-Type	application/json
Controller	MoodleRestController.getClassOverview()
Diễn giải	API trả về thống kê tổng quan về số khóa học và số sinh viên.

Bảng 4.37. Bảng mô tả API lấy lần truy cập cuối

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/{studentId}/last-access
Content-Type	application/json
Controller	MoodleRestController.getLastAccess()
Diễn giải	API lấy thông tin lần truy cập cuối của sinh viên theo từng khóa học.

Bảng 4.38. Bảng mô tả API lấy danh sách khóa học

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/courses
Content-Type	application/json
Controller	MoodleRestController.getCourses()
Diễn giải	API trả về toàn bộ danh sách khóa học hiện có trên Moodle.

Bảng 4.39. Bảng mô tả API gửi thông báo qua Moodle

Thuộc tính	Giá trị
Phương thức	POST
Endpoint	/api/moodle/notifications/send
Content-Type	application/json
Controller	MoodleRestController.sendNotification()
Diễn giải	API gửi thông báo tới người học dựa trên dữ liệu rủi ro.

Bảng 4.40. Bảng mô tả API lấy sinh viên chưa nộp bài

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/courses/submissions/missing?courseName={courseName}&activityType=Assignment
Content-Type	application/json

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Control ler	MoodleRestController.getMissingSubmissions()
Diễn giải	API lấy danh sách sinh viên chưa nộp bài trong một môn học.

Bảng 4.41. Bảng mô tả API lọc sinh viên rủi ro theo môn

Thuộc tính	Giá trị
Phương thức	GET
Endpoi nt	/api/moodle/courses/risk- filter?courseName={courseName}&riskLevel=yellow&minAvgGrade=50 &maxInactiveDays=7
Content -Type	application/json
Control ler	MoodleRestController.getCourseRiskFilter()
Diễn giải	API lọc sinh viên có rủi ro trong một khóa học cụ thể.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

Bảng 4.42. Bảng mô tả API lấy sinh viên không hoạt động

Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/students/inactive?inactiveDays=14&classCode={classCode}
Content-Type	application/json
Controller	MoodleRestController.getInactiveStudents()
Diễn giải	API tìm sinh viên không online quá số ngày quy định.

Bảng 4.43. Bảng mô tả API tổng hợp rủi ro theo lớp

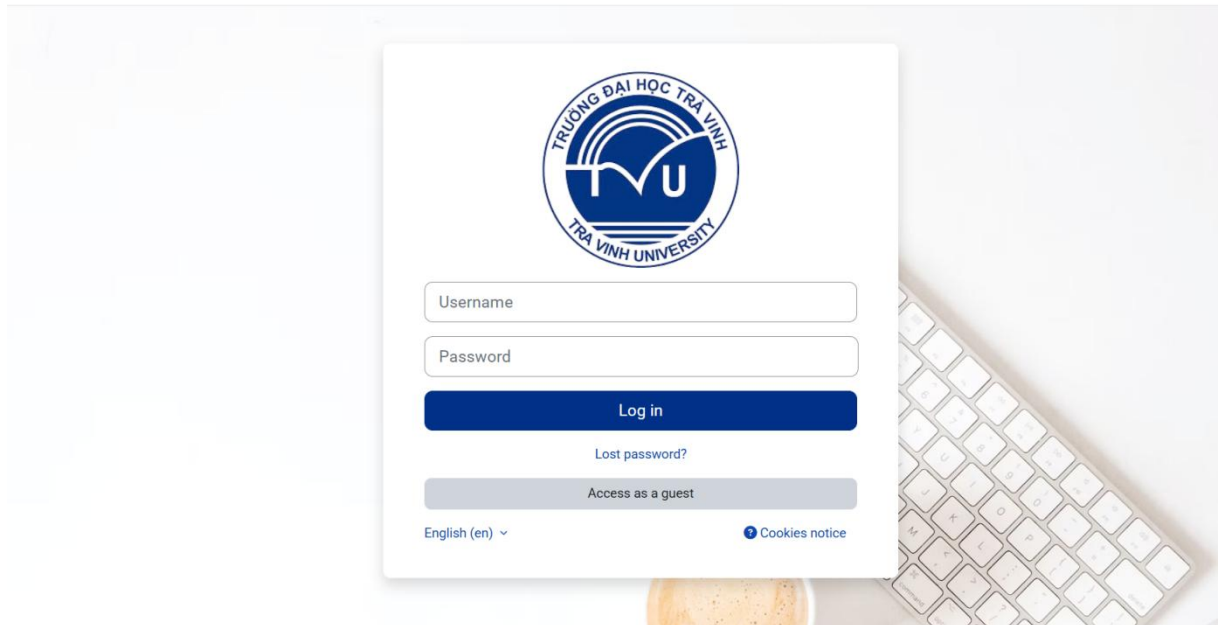
Thuộc tính	Giá trị
Phương thức	GET
Endpoint	/api/moodle/class-risk-summary?classCode={classCode}
Content-Type	application/json
Controller	MoodleRestController.getClassRiskSummary()
Diễn giải	API thống kê sinh viên rủi ro theo lớp hoặc toàn bộ hệ thống.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

4.2. Giao diện hệ thống

4.2.1.1 Chức năng Đăng nhập

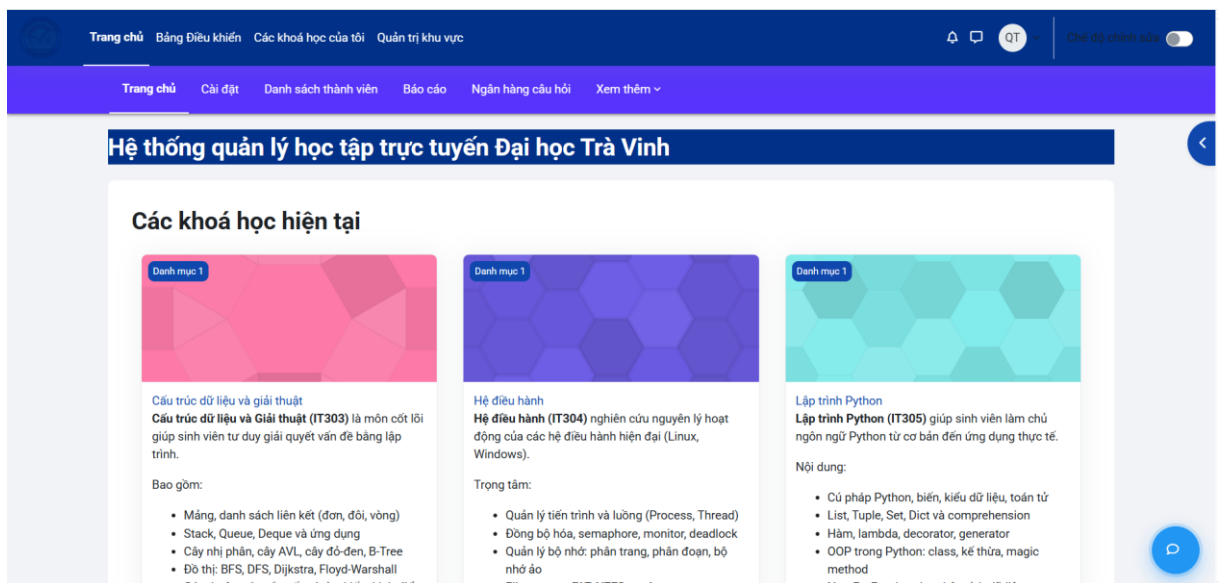
Để bắt đầu sử dụng các tính năng trên hệ thống Chatbot, người dùng bắt buộc phải qua bước đăng nhập vào moodle bằng các quyền cụ thể.



Hình 4.1 Giao diện trang đăng nhập

4.2.1.2 Chức năng Trang chủ

Trang chủ hiển thị thông tin về khóa học đang tham gia, hoạt động gần đây, thông báo mới nhất. Người dùng có thể nhanh chóng truy cập vào các chức năng quan trọng được bố trí trực quan.

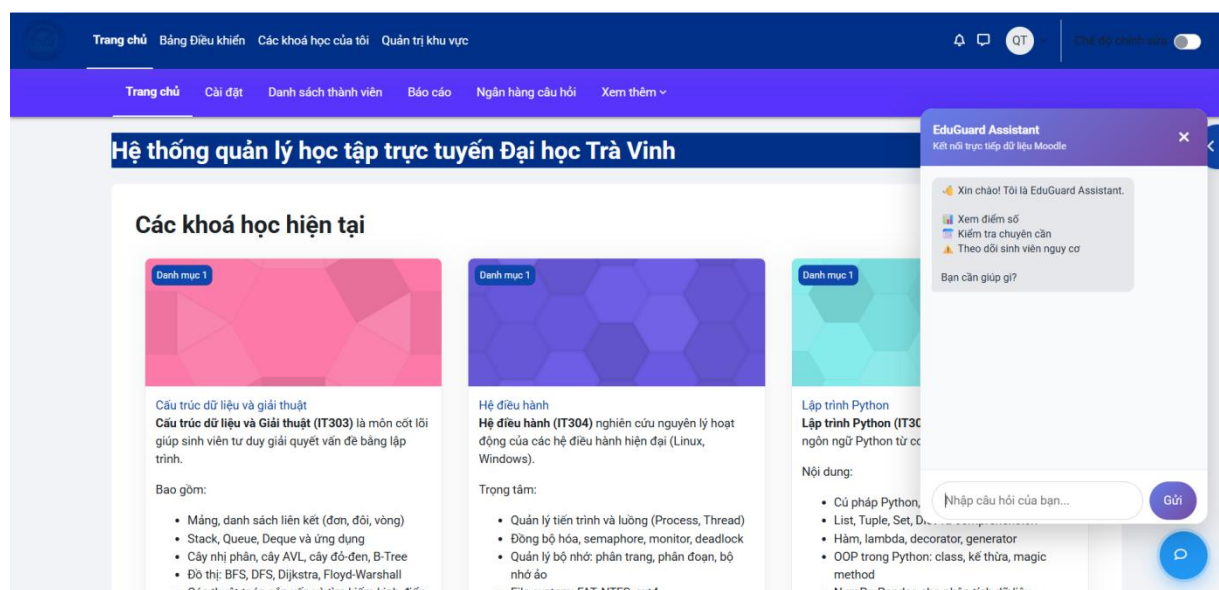


Hình 4.2 Giao diện trang chủ

4.2.1.3 Giao diện Chatbot

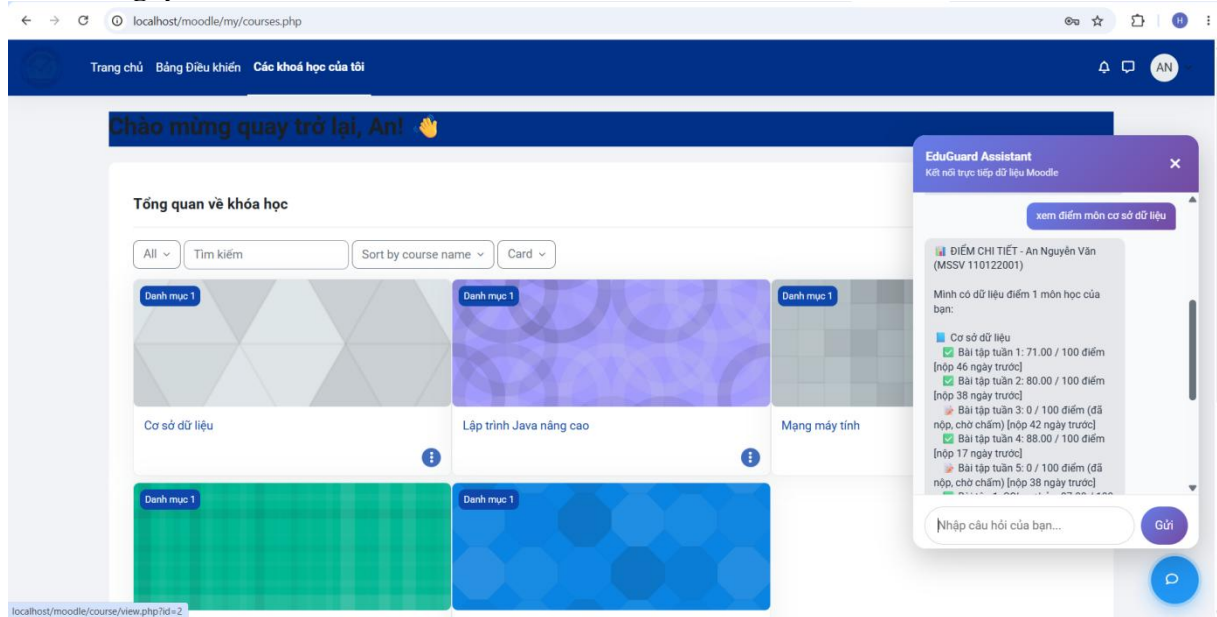
Giao diện chatbot được hiển thị dưới dạng một khung hội thoại cố định ở góc màn hình. Người dùng có thể mở hoặc thu gọn cửa sổ chat tùy theo nhu cầu sử dụng. Khi chatbot được mở, hệ thống hiển thị phần tiêu đề, khu vực hiển thị tin nhắn, ô nhập câu hỏi và khu vực các nút gợi ý thao tác nhanh.

Thiết kế này giúp giao diện tiết kiệm không gian màn hình nhưng vẫn đảm bảo khả năng tương tác thuận tiện. Khi cần sử dụng, người dùng chỉ cần nhấn vào nút mở chatbot để bắt đầu trò chuyện với hệ thống.



Hình 4.3 Giao diện Chatbot

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học



Hình 4.4 Giao diện chatbot hỗ trợ

Sau khi mở giao diện chatbot, người dùng có thể nhập câu hỏi trực tiếp vào ô nhập liệu ở phía dưới cửa sổ chat. Hệ thống hỗ trợ nhập bằng ngôn ngữ tự nhiên, ví dụ như:

- xem danh sách sinh viên nguy cơ,
- tra cứu tình trạng học tập của sinh viên,
- thống kê tổng quan lớp học,
- kiểm tra sinh viên chưa nộp bài,
- xem sinh viên không hoạt động trong thời gian dài.

Khi người dùng nhấn nút gửi hoặc nhấn phím Enter, nội dung câu hỏi được chuyển đến hệ thống xử lý và chatbot sẽ sinh phản hồi tương ứng dựa trên dữ liệu từ cơ sở dữ liệu Moodle và các bảng phân tích nội bộ.

4.2.1.4 Chức năng gợi ý câu hỏi nhanh

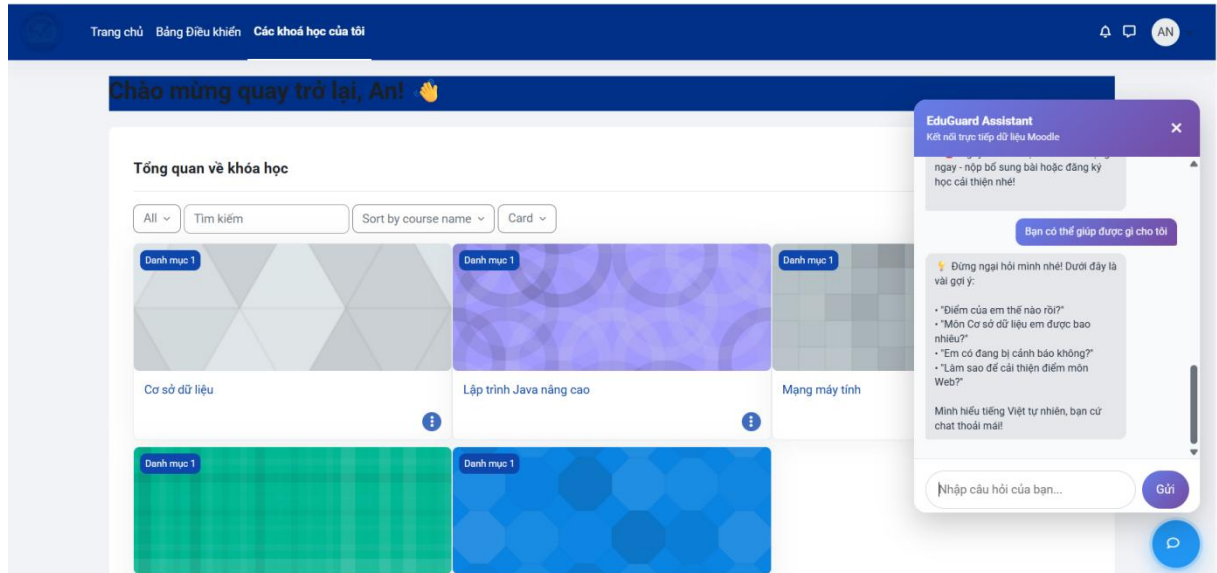
Để hỗ trợ người dùng trong quá trình sử dụng, giao diện chatbot có sẵn một số nút gợi ý câu hỏi nhanh. Người dùng chỉ cần nhấn vào các nút này để gửi ngay những câu hỏi phổ biến như:

- xem sinh viên nguy cơ,
- thống kê tổng quan,

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- kiểm tra tình trạng học tập.

Chức năng này giúp rút ngắn thời gian nhập liệu, đồng thời hỗ trợ người dùng mới dễ dàng làm quen với hệ thống.



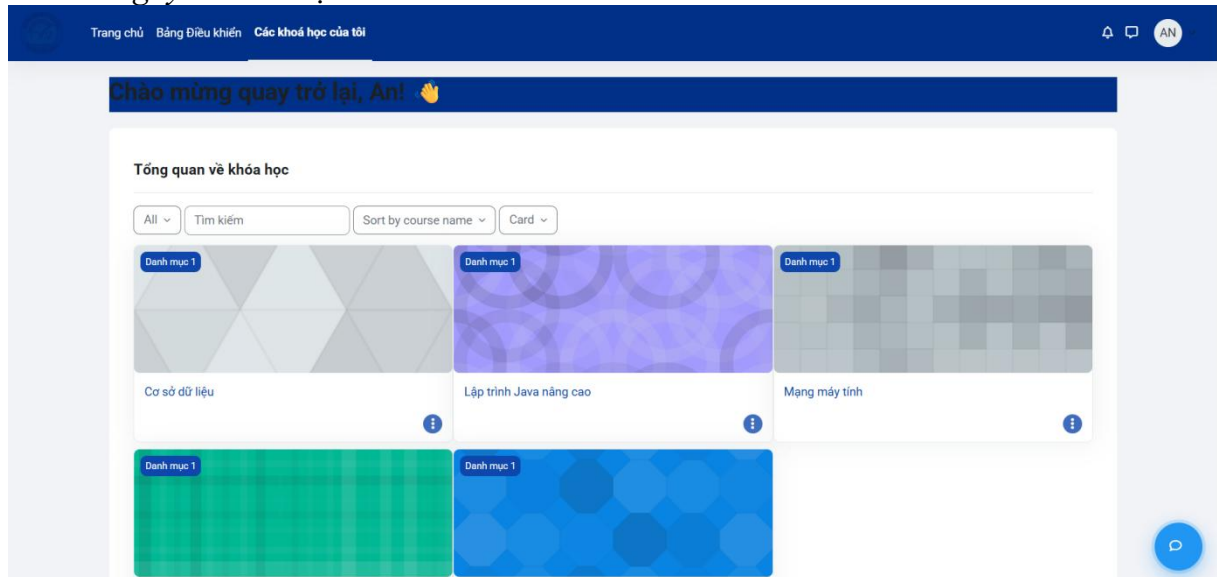
Hình 4.5 Giao diện gợi ý nhanh

4.2.1.5 Chức năng thu gọn và mở rộng giao diện

Giao diện chatbot có thể thu gọn khi người dùng tạm thời không muốn sử dụng. Khi thu gọn, hệ thống chỉ còn hiển thị nút mở chatbot dạng nổi ở góc màn hình. Cơ chế này giúp giao diện không che khuất nội dung chính của trang web nhưng vẫn đảm bảo người dùng có thể truy cập lại chatbot bất cứ lúc nào.

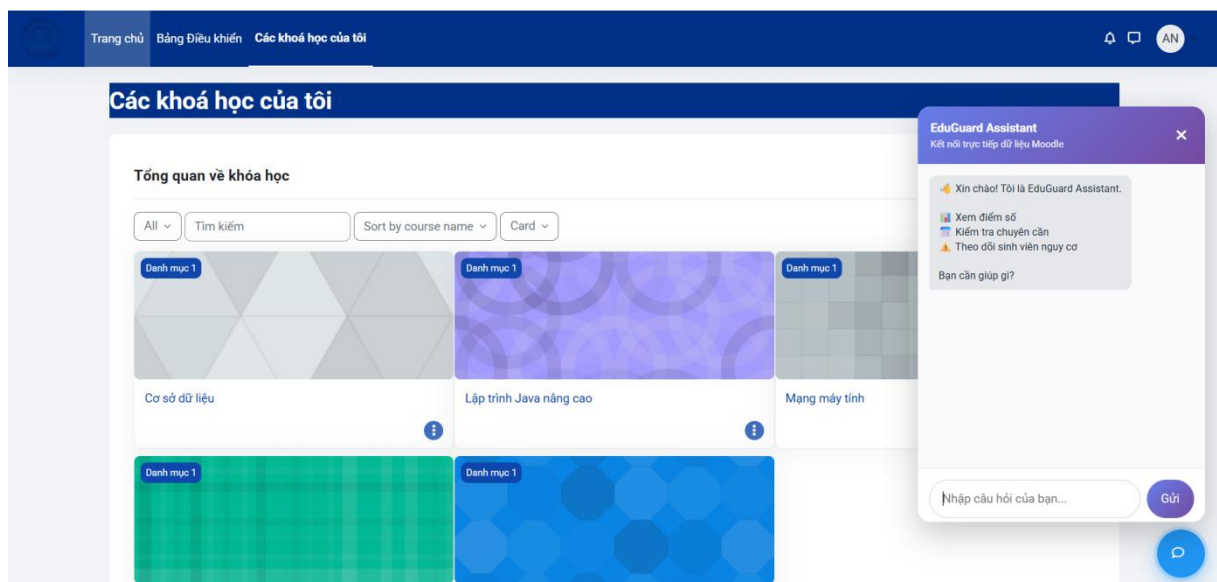
Ngoài ra, giao diện còn được tối ưu để hiển thị tốt trên nhiều thiết bị khác nhau, đặc biệt là trên màn hình nhỏ như máy tính bảng hoặc điện thoại di động.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học



Hình 4.6 Giao diện Chatbot không sử dụng

Giao diện chatbot khi sử dụng.



Hình 4.7 Giao diện chatbot khi sử dụng

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Dự án "Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học" đã được triển khai thành công và hoàn thành các mục tiêu nghiên cứu đề ra. Hệ thống đã chứng minh được tính hiệu quả trong việc giải quyết các thách thức về quản lý chuyên cần, theo dõi dữ liệu học tập và hỗ trợ kịp thời cho sinh viên trong môi trường đại học.

Các thành tựu chính của dự án bao gồm:

- Ứng dụng công nghệ hiện đại: Hệ thống được xây dựng trên nền tảng Java 17 và Spring Boot 3.2.5, đảm bảo tính ổn định, hiệu suất cao và khả năng mở rộng trong môi trường thực tế.
- Tự động hóa cảnh báo: Thông qua việc tích hợp Moodle API và Web Services, hệ thống tự động hóa quá trình thu thập dữ liệu về điểm số, chuyên cần và xác định ngưỡng rủi ro của sinh viên.
- Phân tích thông minh: Sử dụng LLM (OpenAI API) kết hợp với cơ chế LocalIntentMatcher giúp hệ thống hiểu rõ ý định của người dùng, cung cấp phản hồi thông minh và hỗ trợ giảng viên đưa ra các đánh giá khách quan.
- Tối ưu trải nghiệm người dùng: Giao diện trực quan kết hợp với giao thức tương tác qua chatbot giúp giảng viên, cố vấn học tập và sinh viên dễ dàng tra cứu, theo dõi thông tin mọi lúc, mọi nơi.

5.2. Hướng phát triển trong tương lai

Dựa trên nền tảng hiện tại, hệ thống có thể được phát triển theo các định hướng sau nhằm tối đa hóa giá trị và hiệu quả ứng dụng:

- Ứng dụng Trí tuệ nhân tạo (AI) nâng cao:
 - Dự báo rủi ro chính xác: Ứng dụng các thuật toán học máy để phân tích dữ liệu lịch sử, dự đoán sớm sinh viên có nguy cơ thôi học với độ chính xác cao dựa trên sự thay đổi hành vi học tập.
 - Cá nhân hóa lộ trình: Sử dụng AI để phân tích điểm yếu riêng biệt của từng sinh viên, từ đó đưa ra các gợi ý cải thiện lộ trình học tập cá nhân hóa.
 - Nâng cấp đối thoại: Tối ưu hóa mô hình LLM để tạo ra các phản hồi chi tiết, tinh tế và mang tính khích lệ, tạo môi trường tương tác gần gũi như một người trợ giảng chuyên nghiệp.

Xây dựng hệ thống Chatbot hỗ trợ giảng viên theo dõi tiến độ học tập và cảnh báo sinh viên có nguy cơ thôi học

- Mở rộng tính năng và hệ sinh thái:
 - Đa kênh tương tác: Tích hợp chatbot vào các nền tảng phổ biến như Zalo, Telegram hoặc Microsoft Teams để tăng cường khả năng tiếp cận và nhận thông báo tức thời cho người dùng.
 - Hệ thống đánh giá đa chiều: Triển khai thêm các module thu thập phản hồi từ sinh viên về chất lượng giảng dạy, tạo sự kết nối hai chiều giữa nhà trường và người học.
- Tối ưu hóa hiệu suất và trải nghiệm:
 - Kiến trúc phân tán: Tối ưu hóa backend để xử lý khối lượng dữ liệu khổng lồ từ hệ thống Moodle khi quy mô người dùng tăng lên toàn trường.
 - Dashboard cá nhân hóa: Cung cấp bảng điều khiển (dashboard) cho phép giảng viên và cố vấn học tập tự thiết lập các tiêu chí báo cáo riêng theo đặc thù môn học hoặc lớp phụ trách.

DANH MỤC TÀI LIỆU THAM KHẢO

[1] Moodle, “Moodle Documentation: Web Services API,” Moodle, 2026. [Online].

Available: <https://docs.moodle.org/>.

[2] Spring Framework, “Spring Boot Reference Documentation,” 2026. [Online]. Available: <https://spring.io/projects/spring-boot>.

[3] N. Q. Do Thi, “Teaching with the Support of AI Chatbots to Develop Students' Self Directed Learning,” VNU Journal of Science: Education Research, 2026. [Online].

Available: <https://js.vnu.edu.vn/ER/article/view/5404>.

[4] OpenAI, “OpenAI API Documentation,” 2026. [Online].

Available: <https://platform.openai.com/>.